Reliability-Screened Capacity Elasticity under Delayed Telemetry and Actuation: An O-RAN-Motivated Study

FATIMA SUBHI KHALEEL[1], YASSIR AL-AMEEN[1], AND HAMED AL-RAWESHIDY[2]

[1]Department of Communications Engineering, College of Engineering, University of Diyala, Baqubah, Iraq (e-mail: fatima.subhi_eng@uodiyala.edu.iq).
[2]College of Engineering, Design and Physical Sciences, Brunel University of London, Uxbridge, U.K.

CORRESPONDING AUTHOR: HAMED AL-RAWESHIDY (e-mail: hamed.al-raweshidy@brunel.ac.uk).

ABSTRACT Resource elasticity under delayed telemetry can reduce provisioning cost while increasing deadline violations. This study examines that trade-off in an Open Radio Access Network (O-RAN)-motivated, three-stage service chain. A causal controller projects reported queues using applied actions and pending commands, then selects capacity and class reservations on a finite grid. A headroom parameter is calibrated separately under a common classwise mean-miss target of 0.1%. The experiment comprises 336 calibration and 840 held-out executions across seven settings, with twenty independent test seeds per setting. Comparisons include fixed capacity, target-utilization sizing, a queue-weighted cost controller and component ablations. Three fixed-reference comparisons satisfy the complete descriptive reliability screen and reduce mean incremental cost by 35.61–37.73%. Markov-modulated traffic, compound arrivals and fifty-slot actuation expose failures of the selected policy. Derived sensitivity relations explain how headroom and demand uncertainty affect surrogate feasibility, while paired intervals quantify observed cost differences. The results establish conditional advantages within a synthetic packet-service model. They do not certify rare-event reliability or validate a physical O-RAN implementation.

INDEX TERMS O-RAN, software-defined networking, delayed actuation, telemetry, reliability screening, resource elasticity, reproducible simulation.

I. Introduction

An elastic resource controller acts on two different timelines. Its queue report describes a past state, whereas its next command changes a future service opportunity. Between those instants, earlier commands can already have altered capacity. Open Radio Access Network (O-RAN) exposes programmable control across disaggregated components [1]. Software-defined networking (SDN) separates control from forwarding [2], and virtualization makes resource provisioning an explicit orchestration decision [3]. These features motivate accounting for the gap between observing, requesting and activating resources. They do not make those three events simultaneous.

The resulting ambiguity is operationally important. A controller extrapolating an old service rate may request capacity that is already scheduled to arrive. Conversely, a controller that assumes immediate activation may underestimate the workload accumulated before its command becomes effective. Information-age analysis characterizes observation freshness [4], networked control incorporates delayed feedback [5], and predictive control uses models of future action effects [6]. The unresolved question for the present experiment is narrower: how much does explicitly retaining the known action history contribute to the cost–deadline trade-off of an otherwise simple allocator?

That question requires a reliability-aware comparison. Reducing active capacity almost automatically reduces a capacity-based cost, but can also increase deadline misses. A lower cost alone is therefore not evidence of a better allocator. The experiment separates three objects: the stochastic design objective, the finite-grid proxy used to issue commands, and the empirical screen used to interpret measured outcomes. This separation prevents a surrogate-feasible decision or a favorable average cost from being mistaken for a packet-level guarantee.

The study considers two packet classes traversing three serial service stages, motivated by access, transport and compute domains. The controller receives one delayed stage report per control epoch, projects workloads through already applied and pending commands, and chooses expansion and class reservations. A headroom parameter is selected on calibration seeds and locked before testing. All stages use abstract packet-service units. Physical radio channels, pro-

cessor execution times and O-RAN interface interoperability are outside the implemented model.

Three research questions organize the analysis. First, can the calibrated controller reduce cost against a preprovisioned reference while both pass the same classwise reliability screen? Second, is the ledger useful at fixed headroom, and how much of its cost difference comes from capacity versus switching? Third, do shifted arrival patterns, bottleneck location and command delay reveal a gap between proxy feasibility and measured deadline risk?

The contributions are an executable action-ledger controller, an explicit calibration-and-comparison protocol, and a reproducible account of its useful and failing regimes. The mathematical analysis connects exact age-resolved service, delayed command activation and deadline-capacity limits to the implemented approximation. Integer reservation and sensitivity relations explain its ranking rule. The numerical analysis covers 336 calibration and 840 held-out executions, with matched traces, component ablations and seed-level uncertainty. Additional diagnostic decompositions use these saved executions without changing parameter selection or the original qualification rule. Together, these elements isolate the value of known-action replay and identify the workload and actuation regimes in which its empirical cost advantage satisfies the common reliability screen.

The remainder develops the literature-based comparison boundary, the service and information model, the controller and its analytical limits, and the experimental evidence. Successful comparisons and counterexamples receive separate treatment before the deployment limitations are discussed.

II. Related Work and Comparison Boundary

OrchestRAN chooses intelligence algorithms and their execution locations [7], while quality-driven intelligence orchestration deploys and shares near-real-time control applications (xApps) across network services [8]. Their decision objects are not the packet-stage capacities used here. Joint cloud, fronthaul and radio orchestration already treats cross-domain resource coupling [9]. The present study focuses on replaying already issued commands when projecting stale queue measurements, and evaluates this mechanism through a matched ablation.

Implemented virtualized radio access network (vRAN) systems provide a stronger physical basis than homogeneous queues. vrAIn learns radio/computing allocation [10], Nuberu restructures processing under compute deficits [11], and Concordia schedules shared computing against processing deadlines [12]. Online-learning base-station allocation [13] and delay-aware multi-timescale orchestration [14] establish adaptation and delay-aware allocation as foundations for resource control. Proactive workload orchestration [15] and learned functional-split selection [16] also address anticipation and reconfiguration. None of their reported performance numbers can be compared directly with an abstract service-unit cost.

Radio-intelligence-controller allocation [17], multivariate probabilistic forecasting [18], and online-learning energy-aware allocation [19] differ in both prediction machinery and objective. The predictor here holds reported ingress and base capacity constant. Its added information is the controller's own action history, not a learned model of future traffic. This permits an inspectable ablation, but leaves uncertainty from workload evolution unresolved.

Conflict management studies interactions between xApps [20]. In contrast, this paper tracks sequential commands from a single controller. Service-enabled sub-slice isolation [21] and RadioSaber's channel-aware customizable schedulers [22] also provide capabilities that quota borrowing does not establish. EdgeRIC demonstrates control in an implemented low-latency system [23]. Its evidence motivates examining delay, but does not justify assigning a measured real-time meaning to the synthetic slot used below.

ColO-RAN [24], OpenRAN Gym [25], a softwarized prototyping platform [26] and FlexRIC [27] offer programmable experimental foundations. ns-O-RAN [28], Colosseum [29] and the fifth-generation (5G) cellular simulator 5G-LENA [30], including its calibration studies [31], identify possible routes to higher-fidelity validation. Validation on these platforms remains outside the present simulation experiment.

Queue-based stochastic optimization [32], deterministic network calculus [33] and stochastic network calculus [34] address distinct types of guarantees. Analyses of interleaved weighted round-robin [35], deficit round-robin [36] and packet-level delay bounds [37] depend on precise arrival and service assumptions. The workload score in this paper is not a service curve, a stability proof or a probabilistic delay bound. Verification of its implementation is also different from physical validation of the abstraction [38].

The executed comparator families are deliberately transparent. Kubernetes Horizontal Pod Autoscaling (HPA) supplies the target-utilization and downscale stabilization ideas [39]. Drift-plus-penalty (DPP) scheduling supplies the queue-weighted resource-cost trade-off [40]. Both implementations below are adaptations, with changed metrics and timing, rather than reproductions of the original systems or their guarantees. Delay-Tolerant Constrained Online Convex Optimization (DTC-OCO) studies delayed feedback with constraint and regret analysis [41]. Delayed activation of immutable integer commands is a different decision setting, so DTC-OCO is positioning literature only.

Table 1 makes the comparison boundary explicit. The empirical gap addressed here is the joint accounting of stale reports, known pending commands and a common reliability screen in a reproducible queue experiment. The table distinguishes these decision mechanisms and validation settings from the comparator adaptations executed here.

III. System Model and Problem Definition

The comparison boundary above motivates a model that separates physical packet evolution from the information

TABLE 1. Study-level comparison of decision scope and evidence. Positioning studies are not executed benchmarks.

Study	Controlled object and objective	Timing or uncertainty treatment	Relationship to this experiment
[9] (2024)	Joint cloud, fronthaul and radio resources with an energy objective	Cross-domain resource coupling in cell-free massive multiple-input multiple-output (MIMO)	Joint allocation is established. The present abstract cost is not measured energy.
[8] (2025)	Placement and sharing of xApp-based services	Quality-driven orchestration of network intelligence	A different decision object from integer packet-stage capacities.
[13] (2025)	Resource thresholds balancing virtualized base-station performance and energy	Online learning under changing operating conditions	Formal learning analysis does not transfer to the finite-grid workload heuristic here.
[14] (2025)	Radio-resource allocation for delay-sensitive services	Near-real-time latency modeling and real-time queue adaptation	A closer delay-aware reference, but radio-resource and violation models differ. No reproduced performance comparison.
[18] (2026)	Multivariate probabilistic resource-demand forecasts	Learned predictive distributions in a containerized non-real-time control application	Forecasting richer future demand differs from replaying known commands with held rates.
[19] (2025)	Energy-efficient O-RAN resource allocation	Online learning	Distinguishes learned adaptation from fixed, separately calibrated headroom.
[20] (2025)	Conflict management across xApps	Interactions between control applications	Here, reordering concerns commands from one controller, not competing xApps.
[41] (2021)	Constrained online network-resource decisions	Delayed feedback and regret analysis	Feedback delay is distinct from immutable, delayed integer activation.
[11], [12] (2021)	Shared vRAN computation and processing deadlines	Implemented processing and scheduling mechanisms	Stronger physical evidence. No equivalence to the synthetic service chain is asserted.
[39], [40]	Utilization-based sizing and queue-weighted cost	Stabilization or queue-dependent resource decisions	Adapted comparator families executed on the same traces, grids and delays.
Present study	Capacity and class shares under a mean-miss screen	Stale reports, issued-command replay and locked headroom	336 calibration and 840 held-out executions. Conditional savings and failure regimes, without physical validation.

available when a command is issued. The following formulation first specifies service and packet ages, then delayed observations and actions, and finally the cost and reliability objective.

A. Service chain and scheduling

Figure 1 identifies the controlled stages and information paths. Table 2 summarizes the principal symbols and units.

Let $i \in \{1,2,3\}$ index stages, $s \in \{0,1\}$ index classes and $t \in \{0,\dots,T-1\}$ index discrete slots. A packet has a class s, birth slot b and absolute deadline $b + D_s - 1$. A packet completed in slot t has latency $t - b + 1$ and is late if that value exceeds D_s. Let $Q_{is}(t)$ denote the queue at the beginning of slot t, before external arrivals, and let $A_s(t)$ denote newly generated packets. Service $S_{is}(t)$ uses total integer capacity $C_i(t) = c_i(t) + e_i(t)$, comprising base capacity c_i and active paid expansion e_i. The implemented dynamics are

$$Q_{1s}(t+1) = Q_{1s}(t) + A_s(t) - S_{1s}(t), \tag{1}$$

$$Q_{is}(t+1) = Q_{is}(t) - S_{is}(t) + S_{i-1,s}(t), \quad i = 2,3. \tag{2}$$

Stage 1 can serve new arrivals immediately. Stages 2 and 3 serve before their upstream neighbors, so a packet cannot traverse two stages in one slot. Each packet requires one service unit at every stage. Class queues are first-in, first-out (FIFO). There are no drops, retransmissions, alternate routes or finite-buffer admission decisions.

At each slot the engine applies due commands, delivers due reports, inserts external arrivals, requests a report and computes a command if the slot is a control epoch, serves stages in order $3, 2, 1$, and updates measurement histories. This ordering fixes otherwise consequential ambiguities at equal timestamps. Report snapshots are taken after external arrivals and before service. No new snapshot is visible to the controller before its configured delivery time.

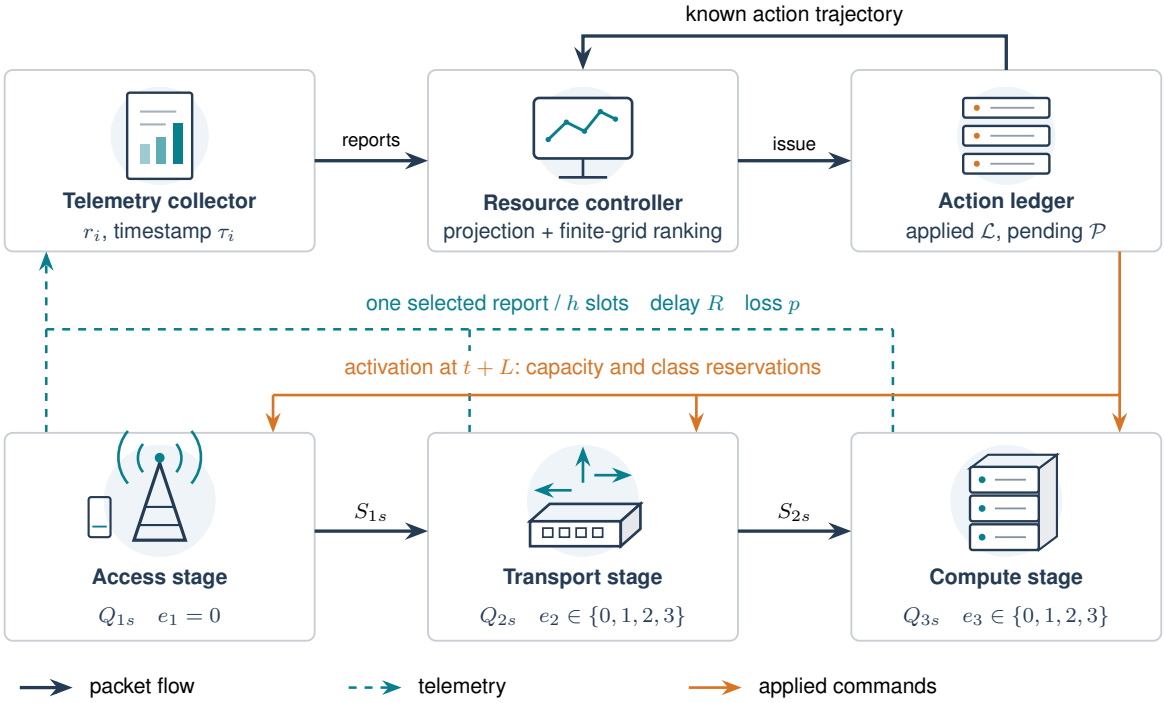


FIGURE 1. **Equipment-inspired view of the three-stage service model and its control loop. Icons identify access, transport and compute roles. They do not imply implemented radio hardware or O-RAN interfaces.**

TABLE 2. **Notation and units. All capacities and workloads are abstract.**

Symbol	Definition
Q_{is}, A_s, S_{is}	Queue, external arrivals and service counts, in packets
c_i, e_i, C_i	Base, expansion and total capacity, in packets per slot
D_s, b, T	Class deadline, birth slot and run length, in slots
τ_i, Δ_i	Timestamp of latest delivered stage report and its age $t - \tau_i$
$\bar{a}_{is}, \bar{\mu}_{is}, v_{is}$	Historical mean ingress, mean service and variance of net ingress
h, W, R, L, p	Control interval, history window, report delay, activation delay and report loss probability
$\alpha, k, \mathcal{X}$	Class-0 share, share-grid index and candidate decision set
$\mathcal{L}, \mathcal{P}$	Applied-action ledger and pending-command list
$\widehat{q}_{is}, d_{is}, \rho_s$	Projected pre-service queue, demand proxy and normalized delay score
η, ϵ, δ	Headroom factor, absolute mean-risk target and paired miss margin
X_{isa}, Y_{isa}	Age-resolved queued and served packets
$m_i, \sigma_{isa}, \omega_{sa}$	Minimum remaining service slots, structural slack and command-relative slack
M_s^{irr}	Cohort packets already unable to meet their deadline
J, V_s	Full-run incremental cost and cohort deadline-miss fraction

For weights $(\alpha, 1-\alpha)$, integer reserved service $g_{is}(C_i, \alpha)$ is obtained by flooring proportional quotas and assigning the remainder to the largest fractional part, with deterministic class-index tie breaking. Thus $\sum_s g_{is} = C_i$. The local scheduler first reserves

$$u_{is} = \min\{B_{is}, g_{is}\},$$
$$B_{1s} = Q_{1s} + A_s, \quad B_{is} = Q_{is} \ (i > 1). \tag{3}$$

Remaining units are borrowed by the eligible class whose next packet has the earliest absolute deadline, with class-index ties. This is deadline-based borrowing after reservations, not unrestricted earliest-deadline-first service. It ensures $\sum_s S_{is} = \min\{C_i, \sum_s B_{is}\}$.

B. Packet ages and deadline accounting

Queue length alone does not identify the number of packets approaching their deadline. Introduce $X_{isa}(t)$, the number of class-s packets at stage i whose age is $a = t - b$, measured after external arrivals and before service in slot t. Let $Y_{isa}(t)$ count the packets of that age served at that stage in that slot. Both variables are nonnegative integers. This representation retains the birth timestamps already present in the simulator

and does not add a new measurement to the controller. Its aggregation is

$$B_{is}(t) = \sum_{a\geq 0} X_{isa}(t), \quad S_{is}(t) = \sum_{a\geq 0} Y_{isa}(t), \\ 0 \leq Y_{isa}(t) \leq X_{isa}(t). \tag{4}$$

Here $B_{1s} = Q_{1s} + A_s$ and $B_{is} = Q_{is}$ for $i > 1$, consistent with the earlier pre-arrival queue convention. With initially empty queues, the exact age dynamics are

$$X_{1s,0}(t+1) = A_s(t+1),$$
$$X_{is,0}(t+1) = 0 \quad (i > 1), \tag{5}$$
$$X_{1s,a+1}(t+1) = X_{1sa}(t) - Y_{1sa}(t), \tag{6}$$
$$X_{is,a+1}(t+1) = X_{isa}(t) - Y_{isa}(t) + Y_{i-1,s,a}(t), \quad i = 2, 3. \tag{7}$$

All survivors age by one slot. Upstream departures become downstream work for the next slot because service is executed in reverse stage order. Summing these equations over age recovers the aggregate queue recurrences. In particular, no packet can traverse two stages in a single slot.

The service constraints combine the stage budget with within-class FIFO:

$$\sum_{s,a} Y_{isa}(t) = \min\left\{ C_i(t), \sum_{s,a} X_{isa}(t) \right\}, \tag{8}$$

$$Y_{isa}(t) > 0 \implies \sum_{v>a} \left[X_{isv}(t) - Y_{isv}(t) \right] = 0. \tag{9}$$

Equation (9) requires every older packet of the same class to be served before a younger age group receives service. Between classes, reserved quotas and earliest-deadline borrowing still determine the totals S_{is} through the scheduler already specified. Thus (8)–(9) are necessary service constraints, not a replacement scheduling optimization with freely selectable departures. Late packets remain queued and consume service. Truncating all ages at D_s would therefore change the model.

Let $\chi(t, a) = \mathbf{1}\{b_{\min} \leq t - a < b_{\max}\}$ identify the measured birth cohort, with $b_{\min} = 300$ and $b_{\max} = 3300$. The indicator $\mathbf{1}\{\cdot\}$ is one when its condition holds and zero otherwise. For $N_s > 0$, the deadline-miss fraction has the equivalent exact expression

$$V_s = 1 - \frac{1}{N_s} \sum_{t=0}^{T-1} \sum_{a=0}^{D_s-1} \chi(t, a) Y_{3sa}(t), \\ N_s = \sum_{t=b_{\min}}^{b_{\max}-1} A_s(t). \tag{10}$$

When $N_s = 0$, set $V_s = 0$. Completion latency is $a + 1$, so the upper limit $D_s - 1$ includes completion exactly at the deadline. Every cohort packet contributes once to either the on-time count or its complement. The tail extends beyond all cohort deadlines, making that complement identical to late completions plus unfinished packets in (22).

This identity also exposes an information loss in the workload proxy. Two states with the same B_{is} and service reservations can have different X_{isa} and hence different attainable on-time counts. At the last stage, one packet of age $D_s - 1$ can still finish on time if served immediately, whereas a packet of age D_s cannot. Their queue lengths are identical. Aggregate feasibility therefore cannot be sufficient for deadline feasibility without additional age information or assumptions.

The age state also gives a sharper necessary deadline condition than queue length alone. A packet at stage i requires at least $m_i = 4 - i$ further service slots because adjacent stages cannot serve it in the same slot. Define its structural slack and the already-unrecoverable measured workload by

$$\sigma_{isa}(t) = D_s - a - m_i, \tag{11}$$

$$M_s^{\text{irr}}(t) = \sum_{i=1}^{3} \sum_{a\geq 0} \chi(t, a) X_{isa}(t) \mathbf{1}\{\sigma_{isa}(t) < 0\}. \tag{12}$$

Even unlimited capacity cannot recover a packet counted by $M_s^{\text{irr}}(t)$ under the specified event order. Each packet occupies only one stage at that instant. Consequently, for $N_s > 0$,

$$V_s \geq \frac{\max_{0\leq t<T} M_s^{\text{irr}}(t)}{N_s}. \tag{13}$$

The maximum, rather than a sum over time, avoids counting the same packet repeatedly. The condition follows directly from the age recurrences. It is checked through independent packet replay rather than estimated from aggregate run records, which do not retain every $X_{isa}(t)$. Packet-level service assumptions likewise determine the applicability of established delay bounds [37], [35].

The currently issued command has an additional timing restriction. Put $\omega_{sa}(t) = D_s - a - L_t$. When $\omega_{sa}(t) \leq 0$, the last permissible completion slot $t + D_s - a - 1$ precedes activation at $t + L_t$. Thus the new command cannot affect that packet's on-time outcome. Earlier active or pending commands may still serve it successfully. This command-specific restriction does not imply that all such packets must miss their deadlines.

C. Telemetry, actuation and switching cost

Every $h = 5$ slots, one stage is selected for a report. Its payload is

$$r_i = (q_i, \bar{a}_i, \bar{\mu}_i, v_i, \bar{c}_i, \tau_i), \tag{14}$$

where q_i is the pre-service queue, $\bar{c}_i$ is the base capacity at the snapshot, and the rate statistics use the preceding $W = 20$ slots. Specifically, v_{is} is the population variance of stage ingress minus service over that window. Empty histories use fixed initialization values listed in the experiment setup. Each request is independently lost with probability p or delivered after R slots. A report older than the stored timestamp is discarded.

The common risk-based report selector uses

$$\widetilde{q}_{is}(t) = [q_{is} + \Delta_i(\bar{a}_{is} - \bar{\mu}_{is})]^+, \tag{15}$$

$$z_i(t) = \sum_s \frac{\sqrt{\Delta_i v_{is} + 1}}{D_s}\left(1 + \frac{\widetilde{q}_{is}(t)}{\max(1, \bar{c}_i)}\right) + 0.01\Delta_i. \tag{16}$$

The stage with greatest z_i is selected, breaking ties by stage index. The final age term is a heuristic priority term, not a bounded-starvation proof. Different methods may select different stages even though their request counts and request-index loss flags are identical.

Figure 2 relates measurement age, report delivery and command activation. A decision at time t creates an immutable command $(t+L, x, w)$ containing an expansion vector and stage-wise class weights. All due commands execute. There is no cancellation, acknowledgment loss or automatic supersession. Constant-delay runs preserve issuance order. In the variable-delay setting, L is announced for each issued command, activations may reorder, and commands due in the same slot execute in issuance order. Future delay draws remain unavailable. Initial fixed capacity is preprovisioned, whereas elastic methods start with zero expansion. Fixed methods still update class shares through delayed commands.

Index issued commands by k, with issue slot t_k, announced delay L_k, activation slot $d_k = t_k + L_k$, expansion x_k and stage-weight matrix W_k. In the proposed controller, each row of W_k is $(\alpha_k, 1-\alpha_k)$. Let command $k = 0$ denote the initialized action, with an activation time before the run. The action active after all due commands in slot t is characterized by

$$\begin{aligned} k^*(t) &= \arg\max_{k: d_k \le t}{}_{\text{lex}} \; (d_k, k), \\ e(t) &= x_{k^*(t)}, \qquad W(t) = W_{k^*(t)}. \end{aligned} \tag{17}$$

The lexicographic maximum first selects the latest activation time and then the latest issue index among simultaneous activations. It is not the latest issue index across all commands. Consequently, an older command with a larger delay can overwrite a newer command that became active earlier.

For the ordered list of commands activating in slot t, write $\mathcal{K}_t = (k_{t,1}, \ldots, k_{t,n_t})$, ordered by issue index. Define $e^{t,0} = e(t-1)$ and $e^{t,j} = x_{k_{t,j}}$ for $j \ge 1$, using the initialized expansion for $e(-1)$. The exact per-slot and full-run costs are

$$j_t = \sum_i e_i(t) + \kappa \sum_{j=1}^{n_t} \|e^{t,j} - e^{t,j-1}\|_1, \tag{18}$$

$$J = \sum_{t=0}^{T-1} j_t. \tag{19}$$

An empty activation list contributes zero switching cost. The triangle inequality gives

$$\sum_{j=1}^{n_t} \|e^{t,j} - e^{t,j-1}\|_1 \ge \|e(t) - e(t-1)\|_1. \tag{20}$$

Thus charging only the net slot change is a lower bound on the implemented switching charge. For example, simultaneous changes $(0,0,0) \to (0,3,0) \to (0,0,0)$ incur six switching units, or cost 1.5 at $\kappa = 0.25$, although the net change is zero. This distinction matters in J, where multiple commands can activate together. Equations (17)–(20) explain why a penalty relative to the previous requested action cannot equal realized switching cost in every execution.

D. Objective and information-constrained formulation

The incremental resource objective is

$$J = \sum_{t=0}^{T-1} \sum_i e_i(t) + \kappa \sum_{a \in \mathcal{A}_T} \left\| x_a - e_a^- \right\|_1, \qquad \kappa = 0.25. \tag{21}$$

The norm $\|z\|_1 = \sum_i |z_i|$ sums componentwise expansion changes for a vector z. The set $\mathcal{A}_T$ contains all command activations within the run, x_a is the activated expansion, and e_a^- is expansion immediately before that activation. Every activation is charged, even when several occur in one slot. The objective includes warm-up and tail capacity, charges actual changes, and excludes base-resource costs. Initial provisioning incurs no switching charge. Units are abstract resource cost units, not money, energy or measured power.

For the measured birth cohort $\mathcal{C}_s$, let $N_s = |\mathcal{C}_s|$ and let ℓ_p be completion latency. The miss fraction is

$$V_s = \frac{N_s^{\text{late}} + N_s^{\text{unfinished}}}{\max(1, N_s)}. \tag{22}$$

Here N_s^{late} counts completed cohort packets with $\ell_p > D_s$, and $N_s^{\text{unfinished}}$ counts cohort packets unfinished at run end. The tail extends beyond all measured deadlines, so unfinished cohort packets are known misses rather than right-censored successes. Background arrivals continue during that tail. Calibration uses a common 0.1% seed-mean miss screening target for each class. This finite-sample selection rule is not a chance constraint.

Let $\mathcal{I}_t$ contain only delivered reports, previously issued commands, the applied ledger, and the current announced activation delay. A causal policy π maps this information to a command. Let Π be the class of such policies respecting the finite capacity and share grids. For a specified workload law, the design objective can be written as

$$\inf_{\pi \in \Pi} \mathbb{E}_\pi[J] \quad \text{subject to} \quad \mathbb{E}_\pi[V_s] \le \epsilon, \qquad s \in \{0, 1\}. \tag{23}$$

The expectation is over independent complete runs under that law, matching the seed-level estimand rather than treating packets as independent trials. Unobserved packet ages, stochastic ingress and delayed immutable commands make this a partially observed sequential problem. The proposed controller does not solve (23). Instead, Algorithm 1 estimates a future queue, the finite-grid ranking selects a command from that estimate, and Algorithm 2 screens a fixed policy parameter on separate calibration runs. The test screen then evaluates its measured cost and miss fractions under each declared law.

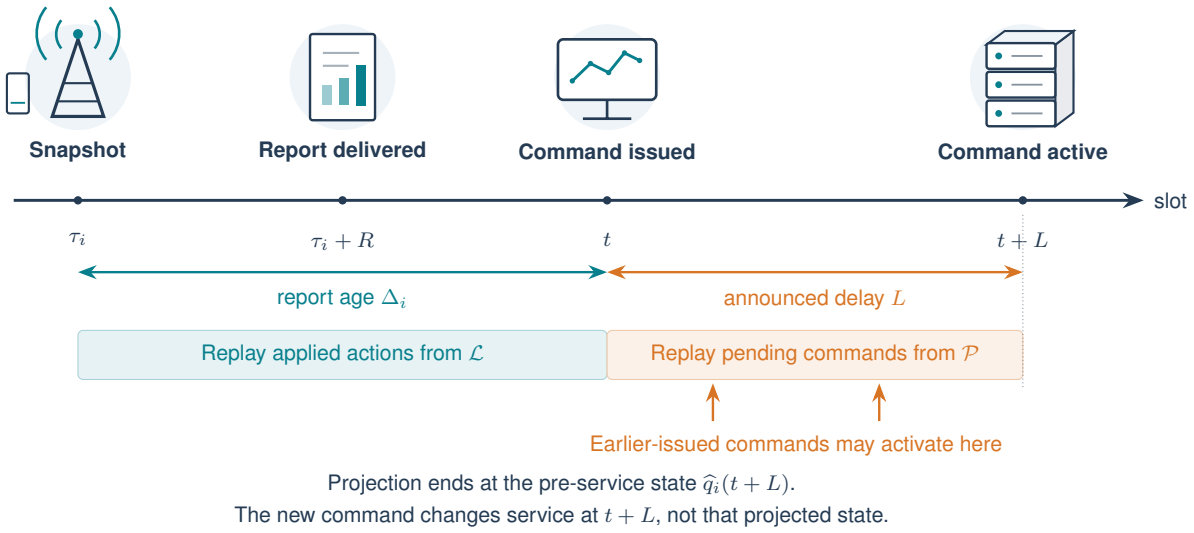


FIGURE 2. Observation-to-activation sequence with explicit equipment roles and action replay. Event positions are schematic. Only already issued commands enter the projection before the new command becomes active.

This separation has two consequences. First, minimizing the one-step proxy cost is not minimizing the full-run objective in (21). A switching penalty relative to the last requested expansion can differ from the actual activation charge when commands reorder. Second, $\rho_s \leq 1$ is only a decision surrogate. It cannot replace the constraint on V_s in (23). The model, algorithms and experiments consequently address three different objects: a design target, an executable approximation, and finite-sample evidence about that approximation.

To make this information restriction explicit, the packet and command descriptions can be combined into a state

$$Z_t = \left(X_t, e_t, W_t, \mathcal{P}_t, \mathcal{R}_t, \mathcal{D}_t, \mathcal{H}_t^{\text{meas}}, \zeta_t\right). \tag{24}$$

Here X_t collects all age bins, $\mathcal{P}_t$ stores issued commands awaiting activation, $\mathcal{R}_t$ stores reports awaiting delivery, and $\mathcal{D}_t$ stores the latest accepted report from each stage. $\mathcal{H}_t^{\text{meas}}$ contains the finite measurement histories, and ζ_t contains the exogenous traffic/capacity state and periodic phase needed for the selected workload law. State-dependent report construction and the event order define a transition map

$$Z_{t+1} = \mathcal{F}_t(Z_t, u_t, \xi_{t+1}), \qquad u_t = \pi_t(\mathcal{I}_t), \quad \pi \in \Pi_{\mathcal{I}}. \tag{25}$$

The random input ξ_{t+1} accounts for exogenous evolution, report loss and any future announced delays. The action u_t specifies the requested command and the stage selected for telemetry, and is null outside control epochs. The information $\mathcal{I}_t$ consists of delivered reports and their timestamps, the applied ledger, previously issued commands and the current announced delay. The set $\Pi_{\mathcal{I}}$ contains policies measurable with respect to this information. It excludes access to the hidden age bins and undelivered reports, even though the simulator retains them for execution.

The objective in (23), together with (5)–(10) and (17)–(25), is therefore a partially observed, integer-valued sequential resource-allocation problem. The actual controller compresses $\mathcal{I}_t$ into projected queue lengths and a finite-grid score. That compression retains known action effects but discards the full age distribution and uncertain future arrivals. The expanded formulation characterizes the underlying problem. The executed algorithm continues to solve its stated per-decision surrogate, using the finite-grid computation described below.

IV. Calibrated Action-Aware Controller

The controller approximates the information-constrained problem using three steps: reconstruct known action effects, rank a finite set of resource choices, and select headroom on separate calibration runs. The projection uses only available information, while the age-resolved model remains the reference for measuring actual deadline outcomes.

A. Projection from the action ledger

At decision time t, the ledger $\mathcal{L}(u)$ records the expansion and weights actually applied at slots $u \leq t$. The list $\mathcal{P}(t)$ contains commands already issued but not yet activated. Their deterministic activation times define a known action trajectory up to $t + L - 1$. This uses no future exogenous arrivals or capacity realizations.

For each stage, initialize $\widehat{q}_i(\tau_i) = q_i$ and hold its latest reported base capacity and mean ingress constant. The fluid recurrence is

$$\begin{aligned}\widehat{q}_{is}(u+1) = [&\widehat{q}_{is}(u) \\ &-F_s\big(\widehat{q}_i(u), w_i(u), \bar{c}_i + e_i(u)\big)\big]^+ + \bar{a}_{is}, \\ &u = \tau_i, \ldots, t + L - 1.\end{aligned} \tag{26}$$

The fluid service operator F first applies integer reservations, limited by the real-valued projected workload, and assigns spare units in descending order of normalized residual workload $(\widehat{q}_{is} - u_{is})/D_s$. Ties use the class index. The projection

does not contain packet ages, so this borrowing rule is only an approximation to the packet scheduler.

Each stage is projected independently using its last reported ingress. Projected upstream departures are not fed into the downstream forecast. This independent projection accommodates asynchronous report timestamps but omits the propagation of future upstream actions between stages. The state at $t+L$ is pre-service and therefore excludes the effect of the new candidate command on that slot's service. Algorithm 1 takes delivered reports and the action ledger as inputs and returns these activation-time queue estimates.

Algorithm 1. Applied-action and pending-command projection

Input: reports r_i, time t, delay L, ledger $\mathcal{L}$, pending list $\mathcal{P}$
Construct known actions for $u = t, \dots, t+L-1$ from $\mathcal{L}(t)$ and $\mathcal{P}$
for each stage $i = 1, 2, 3$ **do**
  Initialize $\widehat{q}_i \leftarrow q_i$ from the latest delivered report
  for $u = \tau_i, \dots, t+L-1$ **do**
    Read applied action from $\mathcal{L}(u)$ if $u \le t$, else the known pending trajectory
    Compute fluid service F using reported base capacity and active expansion
    Update $\widehat{q}_i \leftarrow [\widehat{q}_i - F]^+ + \bar{a}_i$
  end for
end for
Return activation-time pre-service projections $\widehat{q}_i(t+L)$

B. Finite-grid allocation and sensitivity

The bounded-horizon demand proxy is

$$d_{is} = \widehat{q}_{is}(t+L) + h\bar{a}_{is} + \sqrt{h v_{is}} + 1. \tag{27}$$

The square-root term is a dispersion heuristic evaluated over one control interval. It is not a confidence bound on the longer queue-projection horizon. The additive one-packet term is a fixed conservative offset. Both are explicit design choices, not parameters inferred from O-RAN measurements.

The decision grid contains

$$\begin{gathered}\mathcal{X} = \{((0, x_2, x_3), k/20) : \\ x_2, x_3 \in \{0,1,2,3\},\ k \in \{1, \dots, 19\}\}, \\ |\mathcal{X}| = 304.\end{gathered} \tag{28}$$

The same share is applied at all stages. With reported base capacities, candidate integer reservations and headroom η yield

$$\rho_s(x,\alpha) = \frac{1}{\eta D_s} \sum_i \frac{d_{is}}{\max\{10^{-3}, g_{is}(\bar{c}_i + x_i, \alpha)\}}, \tag{29}$$

$$r(x,\alpha) = \max_s \rho_s(x,\alpha), \tag{30}$$

$$c(x) = \sum_i x_i + \kappa \left\| x - x^{\mathrm{req}}_{\mathrm{prev}} \right\|_1. \tag{31}$$

The small denominator floor defines a finite penalty for zero reservations, not guaranteed fractional service. The cost proxy compares with the previous *requested* expansion. It differs intentionally from the realized cost in (21), which uses actual activation changes.

Candidates are ranked lexicographically by

$$\boldsymbol{\gamma}(x,\alpha) = \begin{cases} (0, c, r, \sum_s \rho_s, k), & r \le 1, \\ (1, r, c, \sum_s \rho_s, k), & r > 1. \end{cases} \tag{32}$$

The tuple $\boldsymbol{\gamma}$ is minimized, retaining the earliest enumerated expansion on any remaining tie. Feasible candidates prioritize cost. When none is feasible, the least worst-class ratio is selected. No claim of feasibility is made for the realized packet system from this surrogate decision.

The ranking can be interpreted through its workload-to-service ratio. For a fixed report state, define the unscaled service-time proxy

$$H_s(x,\alpha) = \sum_{i=1}^{3} \frac{d_{is}}{\widetilde{g}_{is}}, \qquad \widetilde{g}_{is} = \max\{10^{-3}, g_{is}(\bar{c}_i + x_i, \alpha)\}. \tag{33}$$

The quantity $\widetilde{g}_{is}$ is the floored reservation denominator used in the implemented ranking. Dividing packet workload d_{is} by reserved packets per slot gives a stage service-time proxy. Adding the stage proxies gives H_s in slots. This algebra does not incorporate the packet ages or joint stage dynamics required for a true end-to-end delay bound. The controller normalizes this proxy by the headroom-adjusted deadline:

$$\begin{gathered}\rho_s = H_s/(\eta D_s), \\ \mathcal{F}(\eta) = \{(x,\alpha) \in \mathcal{X} : \rho_s \le 1\ \forall s\}.\end{gathered} \tag{34}$$

For the same reports and candidate actions, $0 < \eta_1 \le \eta_2$ implies $\mathcal{F}(\eta_1) \subseteq \mathcal{F}(\eta_2)$. Thus reducing η cannot create a newly feasible candidate at that decision state. If both sets are nonempty, the minimum feasible proxy cost cannot decrease when η is reduced. This local statement does not imply monotone realized cost over a closed-loop run, where actions also change future reports and queues.

Holding the projected state and reservations fixed, the corresponding local sensitivities are

$$\frac{\partial \rho_s}{\partial \eta} = -\frac{\rho_s}{\eta}, \qquad \frac{\partial \rho_s}{\partial \widehat{q}_{is}} = \frac{1}{\eta D_s \widetilde{g}_{is}}, \tag{35}$$

$$\frac{\partial \rho_s}{\partial v_{is}} = \frac{\sqrt{h}}{2\eta D_s \widetilde{g}_{is}\sqrt{v_{is}}}, \qquad v_{is} > 0. \tag{36}$$

These derivatives describe the score between changes in integer reservations. They are not derivatives of the selected

action, which can jump between grid points. Equation (35) explains why headroom can exclude cheap candidates, while (36) identifies the greater sensitivity associated with small reservations and short deadlines. At zero variance the square-root term is continuous but its derivative is not finite.

The ranking is discrete, so a smooth derivative alone cannot describe its response to one added capacity unit. For two classes with integer total capacity C and weights $(w_0, w_1) = (\alpha, 1-\alpha)$, define proportional quotas $r_s = Cw_s/(w_0+w_1)$, fractional parts $f_s = r_s - \lfloor r_s \rfloor$ and remainder $\nu = C - \sum_s \lfloor r_s \rfloor$. Largest-remainder allocation gives

$$\begin{aligned} g_0(C,\alpha) &= \lfloor r_0 \rfloor + \nu \mathbf{1}\{f_0 \geq f_1\}, \\ g_1(C,\alpha) &= C - g_0(C,\alpha). \end{aligned} \tag{37}$$

To obtain this expression, floor both proportional quotas and allocate any remaining unit to the larger fractional part, using class index for equality. This explains the plateaus and jumps as α changes. In exact arithmetic, $g_0 = \lfloor \alpha C + 1/2 \rfloor$. Reproduction uses the computed fractional parts instead: binary floating-point values can distinguish a mathematical tie. For example, the saved implementation returns $(3, 2)$ at $C = 5$ and $\alpha = 0.70$. (37) is checked against it throughout the relevant grid.

For fixed demand and a fixed share, compare candidate capacities C_i and $C_i + 1$. Define the corresponding floored denominators as $\widetilde{g}_{is}$ and $\widetilde{g}'_{is}$, as in (33). Their exact contribution to the class-score change is

$$\rho'_s - \rho_s = -\frac{d_{is}(\widetilde{g}'_{is} - \widetilde{g}_{is})}{\eta D_s \widetilde{g}_{is} \widetilde{g}'_{is}}. \tag{38}$$

This follows by subtracting the two reciprocal service terms. An added unit can leave one class's score unchanged if its reservation does not increase. Its effect is amplified when it removes a zero reservation, where the numerical denominator floor is active. Thus a common share and an integer service budget couple the two classes even though the score is computed classwise. The quantity in (38) is a proxy improvement, not an observed change in packet latency.

The score also permits a finite perturbation bound. At fixed candidate reservations, let E^q_{is}, E^a_{is} and E^v_{is} bound the absolute differences between two projected queues, mean-ingress estimates and nonnegative net-flow variance estimates. From (27), the triangle inequality and $|\sqrt{v'} - \sqrt{v}| \leq \sqrt{|v' - v|}$ give

$$|d'_{is} - d_{is}| \leq E^q_{is} + hE^a_{is} + \sqrt{hE^v_{is}}, \tag{39}$$

$$|\rho'_s - \rho_s| \leq B_s := \frac{1}{\eta D_s} \sum_i \frac{E^q_{is} + hE^a_{is} + \sqrt{hE^v_{is}}}{\widetilde{g}_{is}}. \tag{40}$$

If such bounds were available and $\rho_s + B_s \leq 1$ for both classes, the same candidate would remain surrogate-feasible under those perturbations. No such error bounds were calibrated or enforced in the executed controller. The bound characterizes perturbation sensitivity without modifying the executed policy.

Equation (40) connects the sensitivity analysis to the test design. At equal reservations and error magnitudes, the shorter deadline $D_0 = 15$ amplifies score perturbations by $D_1/D_0 = 8/3$ relative to $D_1 = 40$. Meanwhile, the projection horizon is $\Delta_i + L$, but the dispersion allowance uses only $h = 5$. Increasing L therefore extends exposure to unmodeled ingress changes without automatically enlarging that allowance. The ledger removes ambiguity about known actions, not uncertainty about future traffic. This distinction motivates examining both proxy-feasibility frequency and actual misses in the results.

C. Calibration, execution and comparison policies

Let $\eta \in \{0.25, 0.5, 0.75, 1\}$ be a dimensionless headroom parameter. The ratios in the ranking are divided by η, so smaller values impose a stricter surrogate-feasibility threshold. They do not impose a hard constraint on realized packet misses. The control interval, dispersion term and candidate grid remain fixed across the headroom candidates.

For each policy family m and candidate parameter θ, calibration estimates the seed mean of each class miss fraction in two report-quality strata. Let $U_{m\theta cs}$ be its one-sided 95% percentile-bootstrap upper endpoint, where c identifies the stratum and s the class. Selection is

$$\begin{aligned} \theta^*_m &= \arg\min_{\theta \in \Theta_m} \overline{J}_{m\theta} \\ &\quad \text{subject to } U_{m\theta cs} \leq \epsilon \quad \forall c, s, \\ &\quad \epsilon = 10^{-3}. \end{aligned} \tag{41}$$

The finite set Θ_m contains the candidate parameters for family m. Here $\overline{J}$ averages calibration costs across both strata. The threshold is a 0.1% seed-mean miss target, not a per-packet chance guarantee or an O-RAN service-level specification. If a family has no passing candidate, it is explicitly marked calibration-ineligible and its prespecified conservative endpoint is retained for diagnosis only. The proposed policy uses its locked parameter throughout each test run.

The same calibration rule is applied to the following comparison families. The fixed family enumerates all sixteen profiles $(0, x_2, x_3)$ on the same calibration seeds. It starts preprovisioned and retains the earlier drift-based share controller. It does not use future traffic, but its known bottleneck location is used to transpose the selected profile for the compute case. Consequently, the fixed comparison evaluates complete policies, not the isolated effect of capacity switching under identical class shares.

The HPA-inspired adaptation computes a target expansion at each expandable stage using target utilization u:

$$\begin{aligned} b_i(t) &= \left[\left\lceil \frac{\sum_s \bar{a}_{is} + \sum_s q_{is}/h}{u} \right\rceil - \bar{c}_i \right]_0^3, \\ x_i(t) &= \max_{v \in \mathcal{H}_t} b_i(v). \end{aligned} \tag{42}$$

The clipping operator $[z]^3_0$ limits z to $[0, 3]$, and $\mathcal{H}_t$ contains recommendations generated in the last twenty slots. Stage 1 has zero expansion. Candidate utilizations are $0.55, 0.70, 0.85, 0.95$. Queue clearance over h slots is a task-

specific addition to ratio-based sizing. The common finite-grid share ranking with $\eta = 1$ selects weights for this fixed target capacity, using the drift projection rather than the action ledger. This is neither a Kubernetes deployment nor its default timing.

The DPP-inspired adaptation uses a price $v \in \{0.1, 0.3, 1, 3\}$ and selects the candidate minimizing

$$vc(x) - \sum_{i,s} \frac{\widetilde{q}^L_{is} + h\bar{a}_{is}}{D_s} \min\{d^{\text{drift}}_{is}, g_{is}(\bar{c}_i + x_i, \alpha)\}. \quad (43)$$

Here $\widetilde{q}^L_{is} = [q_{is} + (\Delta_i + L)(\bar{a}_{is} - \bar{\mu}_{is})]^+$, and d^{drift} replaces the action-aware queue in (27) with that drift projection. Cost breaks objective ties, followed by share index and enumeration order. Deadline normalization and clipped service rewards are explicit adaptations. No virtual deadline-deficit queues, backpressure differentials or imported Lyapunov guarantee are claimed. A failed calibration for this adaptation is not evidence against the broader DPP method family.

Two internal ablations complete the comparison. *Uncalibrated* uses the action ledger with $\eta = 1$. *No ledger* uses the selected headroom but substitutes $\widetilde{q}^L$ for the action-aware projection. The latter isolates projection effects at a common headroom parameter, not an independently optimized alternative policy family. Algorithm 2 separates the offline selection of parameters from online command execution. The selected parameters remain locked throughout the held-out evaluation.

Algorithm 2. Reliability-screened selection and elastic execution

Input: calibration seeds, parameter grid, ϵ, frozen test settings
for each family and candidate parameter **do**
  Execute both calibration strata and compute classwise upper endpoints
end for, select minimum-cost passing candidate by (41)
Lock selections and record any calibration-ineligible family
at each elastic control epoch t **do**
  Read delivered reports, applied ledger and announced activation delay
  Project queues with Algorithm 1 and compute η-scaled ratios
  Select the minimum-ranked candidate from all 304 possibilities
  Enqueue the command at $t + L$ without cancellation
end, evaluate held-out cost and misses without retuning

V. Formal Properties and Limits

The preceding construction supports exact statements about information use, packet conservation and the finite-grid decision. These properties are followed by deadline-capacity bounds that identify what even a better ranking rule cannot repair once the remaining service opportunities are insufficient.

Proposition 1 (Causality)**.** *For fixed configuration and initialization, every elastic decision at time t is a function only of reports delivered by t, commands already issued and their known activation times. It is independent of unobserved future traffic and base-capacity realizations.*

Proof:
The report selector reads only stored payloads and their timestamps. The ledger contains previously applied commands. The pending list contains commands issued at earlier epochs. Equation (26) uses these actions and holds reported base capacity and ingress fixed. The demand and ranking equations are deterministic functions of that projection. Induction over decision epochs therefore excludes future exogenous trace entries. A future-suffix perturbation test additionally checks this property in the executable implementation. ■

Proposition 2 (Conservation and service bounds)**.** *For every slot, all queues remain nonnegative, stage service does not exceed available capacity, and the total number of generated packets equals completed packets plus packets remaining in the chain. Completed latency is at least three slots.*

Proof:
Reservations are limited by available queue contents. Borrowing only selects a class with a remaining packet and stops at capacity, proving nonnegativity and the service bound. Summing (1)–(2) cancels each internal transfer, leaving external arrivals minus final-stage completions. Induction from empty queues proves conservation. Reverse stage order prevents a packet from receiving two stages of service in one slot, giving the latency bound. ■

Proposition 3 (Finite-grid decision optimality)**.** *The enumerator returns a minimum of (32) on $\mathcal{X}$. If any surrogate-feasible candidate exists, the result is surrogate-feasible and has minimum proxy cost among all such candidates.*

Proof:
All sixteen expansion vectors and nineteen share values are visited. The running incumbent changes only for a strictly smaller lexicographic score. After the last candidate it is a minimum over the complete finite set. The leading score component ranks every feasible candidate ahead of every infeasible candidate, and cost is the first varying component within the feasible subset. This establishes the stated result, not global stochastic-control optimality. ■

Proposition 4 (Actuation irreversibility)**.** *Consider two executions with identical exogenous traces, initial states and all applied actions before slot a. A command that first changes an action at slot a cannot change the on-time-completion indicator of any packet whose absolute deadline is strictly earlier than a.*

Proof:
The deterministic event contract gives identical packet states and service decisions through slot $a-1$ by induction. Every completion opportunity up to the stated packet deadline is therefore identical. A change beginning at a cannot alter whether the packet completed by an earlier slot. ∎

In particular, a decision issued at t with activation $t+L$ cannot rescue a packet with deadline below $t+L$ unless other already-scheduled actions do so. This timing restriction explains why accurate accounting of pending actions cannot remove arbitrarily long actuation latency. Preprovisioning, predictive information before the burst, or a faster local mechanism is required to change those earlier opportunities.

A. Deadline limits and computational cost

Age resolution sharpens the aggregate capacity obstruction. At a pre-service slot t, consider measured-cohort packets currently waiting at stages $j \le i$ whose absolute deadlines are no later than d, with $t \le d < T$. Their count is

$$B_i^{\mathcal{C}}(t,d) = \sum_{j=1}^{i} \sum_{s,a} \chi(t,a)\mathbf{1}\{t-a+D_s-1 \le d\} X_{jsa}(t). \tag{44}$$

Every such packet still needs stage-i service. Because the remaining $3-i$ stages each require a later slot, it must leave stage i no later than $d-(3-i)$. Even if all available service were assigned to these packets, their on-time completion count cannot exceed

$$K_i(t,d) = \sum_{u=t}^{d-(3-i)} C_i(u), \tag{45}$$

with an empty sum interpreted as zero. Let $M^{\mathcal{C}}(t,d)$ denote the eventual number of misses among currently queued cohort packets with deadline at most d. The necessary pathwise inequality is

$$M^{\mathcal{C}}(t,d) \ge \max_{i\in\{1,2,3\}} [B_i^{\mathcal{C}}(t,d) - K_i(t,d)]^+. \tag{46}$$

To prove the inequality, each on-time packet in the stage-i subset must consume one distinct service unit in its stated window. At most K_i such units exist, leaving at least $[B_i^{\mathcal{C}} - K_i]^+$ misses. The maximum combines valid lower bounds without counting the same packet multiple times. The bound relaxes upstream release times, other traffic and reserved shares, so a zero value does not establish feasibility. Unlike a steady-state load condition, it retains residual deadlines and downstream transit time.

For the constant-delay cases, no command issued at or after t changes capacity before $t+L$. Let $e_i^{\mathrm{pend}}(u)$ be the expansion obtained by replaying the applied action and commands already issued before t, and let $e_i^{\max}$ be 0 for stage 1 and 3 otherwise. A control-independent upper envelope on capacity opportunities is

$$\overline{K}_i(t,d) = \sum_{u=t}^{d-(3-i)} \left[c_i(u) + \begin{cases} e_i^{\mathrm{pend}}(u), & u < t+L, \\ e_i^{\max}, & u \ge t+L. \end{cases} \right]. \tag{47}$$

Since $K_i \le \overline{K}_i$, substituting $\overline{K}_i$ into (46) gives a lower bound valid for every subsequent admissible control sequence under that fixed-delay assumption. Future base capacities are not observed by the controller, so this is a pathwise analytical bound, not an implemented online certificate. In J, later-issued commands can have shorter delays. The fixed-L envelope must not be applied to that case.

A necessary condition for all packets in any cohort to meet their deadlines is that each stage has enough cumulative service opportunities within the union of their allowable residence intervals. In particular, if a cohort born in $[a,b]$ has N packets and maximum deadline $D_{\max}$, then

$$N \le \sum_{t=a}^{b+D_{\max}-1} C_i(t) \qquad \text{for every stage } i. \tag{48}$$

Every such packet must consume one unit at every stage within that interval. Violation of (48) therefore precludes universal deadline satisfaction. The condition is loose and not sufficient because it ignores class deadlines, competing packets, stage precedence and service timing.

For disjoint service windows $I_j = [a_j, b_j + D_{\max} - 1]$ containing cohorts of size N_j, the same argument gives the zero-miss necessary cost bound

$$J \ge \sum_i \sum_j \left[N_j - \sum_{t\in I_j} c_i(t) \right]^+. \tag{49}$$

Disjointness prevents counting the same capacity opportunity twice. This lower bound omits switching charges, competing traffic and precedence, so it is not an achievable optimum. It applies to zero misses for those cohorts, not to the relaxed empirical target in this experiment. No approximation ratio or optimality gap is inferred from it.

With two classes and three stages fixed, projection work scales with $\sum_i(\Delta_i + L)$. The grid requires 304 candidate scores, with reusable stage/share calculations. In the general m-stage extension with $E+1$ expansion levels at each expandable stage and K shares, exhaustive selection scales as $O(K(E+1)^{m-1}mS)$ for S classes. Thus the small present grid is not evidence of scalability to many independent resource domains. The reference implementation retains a full applied-action ledger. A production implementation could discard entries older than the oldest stored report, subject to its report delivery contract. No wall-clock real-time feasibility claim is made.

VI. Experimental Method

The experiments test the distinction established by the analysis: a causal, surrogate-optimal decision need not meet packet deadlines. Nominal runs measure cost under the common reliability screen, shifted workloads probe the deadline limits, and ablations isolate the roles of headroom and action memory.

A. Workloads and experimental controls

Each run has 3340 slots: 300 warm-up slots, a 3000-slot measured birth cohort, and a forty-slot tail. Background traffic continues during the tail. The class deadlines are $(15, 40)$ slots. An illustrative mapping of twenty milliseconds per slot gives 300 and 800 milliseconds, not ultra-reliable low-latency communication (URLLC) deadlines. No measured mapping from packet units to physical resource blocks or central processing unit (CPU) cycles is used.

For M and B, a two-state chain begins in the low state. At each slot its low-to-high probability is 0.005 and its high-to-low probability is 0.02. Poisson arrival means in each state equal the corresponding nominal and burst means above. Transport capacity is three in the high state and fluctuating base capacity in the low state. Other stages retain the exogenous base-capacity draws. In B, the number of arriving batches is Poisson with mean one third of the conditional packet rate, and each batch contains three packets. Thus B preserves conditional means but increases conditional variance. These cases alter both burst timing and its correlation with transport capacity. They are synthetic distribution shifts, not independent real-network datasets.

In J, every command delay is sampled uniformly from $\{1, 5, 10, 25, 50\}$ and announced when that command is issued. Controllers do not see future delay draws. All methods share the same delay sequence within a seed. The mean delay is 18.2 slots, so P versus J is not a pure variance comparison at equal mean. Multiple commands can activate in one slot, in issuance order. Commands with longer delays may activate after subsequently issued commands. They are not silently canceled or superseded.

Calibration and test executions follow a fixed local protocol specifying parameter grids, seeds and the calibration-failure rule. Calibration seeds 12000–12005 are distinct from test seeds 13000–13019. The two calibration strata are P and R, each at ten-slot actuation delay. Sixteen fixed profiles plus four candidates for each of the three elastic families give $(16 + 4 + 4 + 4) \times 2 \times 6 = 336$ calibration executions. After selection, seven settings, six methods and twenty seeds give 840 test executions. All six methods use the same seed-specific exogenous trace, the same request-index loss flags and one request every five slots. Trace hashes, admitted cohort counts and delivered-report counts are checked for equality. Different methods can select different report stages, so payloads need not be identical. Fixed provisioning has a larger calibration search budget than each elastic family. Each family has only one selected test configuration.

The prespecified diagnostic fallback parameters are fixed $(3, 3)$, headroom 0.25, utilization 0.55 and DPP price 0.1. They are used only when the corresponding family has no passing calibration candidate, and that status remains visible. Only the 336 calibration and 840 held-out executions contribute to the present analysis.

B. Statistical evaluation and verification

The independent replication unit is a complete seed, not a packet. Results average seed-level miss fractions and costs. Two-sided 95% percentile-bootstrap intervals use 10,000 seed resamples [42]. One-sided upper endpoints use the 95th percentile of 10,000 bootstrap means. Paired comparisons resample within-seed differences. Packet dependence within a run is therefore not mistaken for thousands of independent trials.

For a proposed-versus-reference comparison in setting c, a qualified cost advantage requires: both families passed calibration, both test upper endpoints satisfy $U_{cs} \leq \epsilon$ for both classes, the upper endpoint of each paired miss difference is at most $\delta = 10^{-3}$, and the upper endpoint of the paired cost difference is negative. The margin δ is 0.1 percentage point. This is descriptive noninferiority screening under common thresholds, not equality of reliability or a guaranteed safety claim. Internal ablations are displayed as diagnostic comparisons, not separately calibrated deployment families.

These intervals are unadjusted across settings, classes, candidates and comparators. Multiplicity-control procedures such as [43] address a family of hypothesis tests. They are not applied to the descriptive bootstrap screens reported here, so the intervals do not support familywise confirmatory inference. Calibration selection and the small six-seed calibration sample can overfit the screen. Fresh test seeds mitigate reuse of outcomes but do not remove finite-sample uncertainty or protect against distribution shift.

An all-zero sample gives a zero-width percentile interval. It does not prove zero risk. With zero seeds exhibiting any miss among twenty independent, identically distributed test seeds, an exact one-sided 95% binomial upper bound for the probability that a whole run exhibits any miss is $1 - 0.05^{1/20} = 0.1391$. This is a *run-event* bound, not a packet-miss bound. The all-zero bootstrap result is consequently treated as a descriptive screen, not a rare-event certificate.

Verification links this statistical protocol back to the mathematical model. Independent age-bin reconstruction checks six complete executions and 306 stage/deadline cuts against frozen outcomes. Verification comprises 51 unit-test methods, regression checks, independent packet replay and reconciliation of all 1176 execution records. Tests include uncached finite-grid enumeration, constant delay regression, variable-delay activation order, future-trace exclusion, compound-arrival construction and independent replay under jitter. Six selected full test runs are regenerated and compared exactly to saved records. Mathematical cost definitions include every applied command change, including multiple

TABLE 3. Fixed parameters and complete held-out matrix. Each setting runs six methods on twenty seeds.

Item	Specification
Common model	Three stages, two classes, $h = 5$, $W = 20$, $\kappa = 0.25$
Base capacity	$5 + \xi$ per stage, ξ sampled from $(-1, 0, 0, 1)$
Periodic bursts	Means $(0.9, 2.1)$ outside and $(1.8, 3.8)$ inside $t \bmod 600 \in [180, 290)$
Capacity stress	Stage 2 capacity 3 during $t \bmod 600 \in [170, 310)$
Initialization	Queue $(0, 0)$, ingress $(0.9, 2.1)$, service $(2.5, 2.5)$, variance $(1, 1)$, capacity 5
P	Periodic bursts, transport stress, $(R, p, L) = (3, 0.05, 10)$
M	Markov-modulated Poisson traffic and transport stress, $(3, 0.05, 10)$
B	Markov-modulated batches of three, same conditional means, $(3, 0.05, 10)$
C	Periodic bursts with stress relocated to stage 3, $(3, 0.05, 10)$
R	Periodic transport stress with degraded reports, $(9, 0.15, 10)$
L	Periodic transport stress, long actuation delay, $(3, 0.05, 50)$
J	Periodic transport stress, $(R, p) = (3, 0.05)$, announced $L \in \{1, 5, 10, 25, 50\}$

changes within a slot. Tables and vector plots are built directly from the saved execution records. The same records support the reported tables, intervals and figure data.

VII. Results and Mechanistic Interpretation

The results follow the evaluation protocol in three stages. Calibration and held-out screening first identify admissible cost comparisons. Traffic and delay shifts then expose failures. Finally, ablations connect those outcomes to the capacity, switching and workload terms in the model.

The selected fixed expansion is $(x_2, x_3) = (2, 0)$, the selected headroom is $\eta = 0.75$, and the selected utilization is $u = 0.85$. No DPP-inspired candidate passes both calibration strata, so price $v = 0.1$ remains diagnostic only. Figure 3 reports all candidates, including failed choices. The three lower headroom values pass the empirical calibration screen. The $\eta = 1$ variant does not. Selection minimizes cost among passing grid points, not over an unknown continuous optimum or a confidence-certified feasible set. Table 4 and Figure 4 isolate the headroom sweep on identical calibration seeds. Tightening η contracts the candidate set in (34), which is consistent with higher observed resource cost. The values 0.25 and 0.50 purchase substantially more capacity than 0.75 without improving their already-zero empirical miss endpoints. Those zeros do not establish equal true risk. At $\eta = 1$, reduced provisioning is accompanied by failed endpoints. The trade-off is therefore substantive, not merely a tunable score with no connection to packet outcomes.

Figure 5 renders every part of the unchanged comparison rule. The paired cost condition passes in all seven settings. Reliability, not the sign of the cost difference, determines which savings can be qualified. P, R and J pass every condition. M, B and L fail the proposed absolute-risk screen, while C fails because the fixed reference exceeds its class-1 target. This is why the zero observed misses of the proposed policy in C do not produce a qualified comparison under the declared rule. Table 5 supplies the endpoints behind that decision. In P, mean cost is 4159.85 versus 6680.00 for fixed provisioning. The paired difference is -2520.15, with a 95% interval approximately $[-2593.1, -2447.0]$. Qualified savings are 37.73% in P, 37.10% in R and 35.61% in J. Figure 6 shows why negative cost differences in the other settings must remain unqualified. These comparisons use common thresholds, not proof that the policies have identical underlying reliability.

Figure 7 separates resource cost from each class's miss fraction. Under Markov traffic M, proposed mean misses are 0.0915% and 0.8811%. Under compound batches B, they are 0.3413% and 0.1297%. Their upper endpoints fail the target. Mean-matched burst states therefore do not preserve the periodic calibration result. Both the duration of unfavorable service periods and the arrival process seen by each queue matter. For batch size $b = 3$, conditional arrivals are $A_s = bK_s$ with $K_s \sim \text{Poisson}(\lambda_s/b)$. Hence

$$\mathbb{E}[A_s \mid \lambda_s] = \lambda_s, \qquad \text{Var}(A_s \mid \lambda_s) = b\lambda_s. \tag{50}$$

The compound case changes dispersion without changing conditional mean. However, the reported v_{is} measures net ingress minus service, not arrival variance alone. Reservations and borrowing also change with the resulting queue trajectories. The opposite movements of the two class means between M and B therefore cannot be summarized as a monotone classwise variance effect.

In L, $L = 50$ exceeds both deadlines, and class-1 mean misses reach 8.7846% versus 0.0208% for fixed provisioning. For any packet already present at issue, $\omega_{sa} \le 0$ because $L > D_s$. Its current command therefore cannot change an on-time completion. The irreversibility property explains the timing obstruction: a new command cannot change service before activation. The horizon mismatch in (40) provides an additional reason not to interpret a bounded dispersion term as protection against this delay. J instead mixes shorter and longer delays, has mean 18.2 slots, and announces each

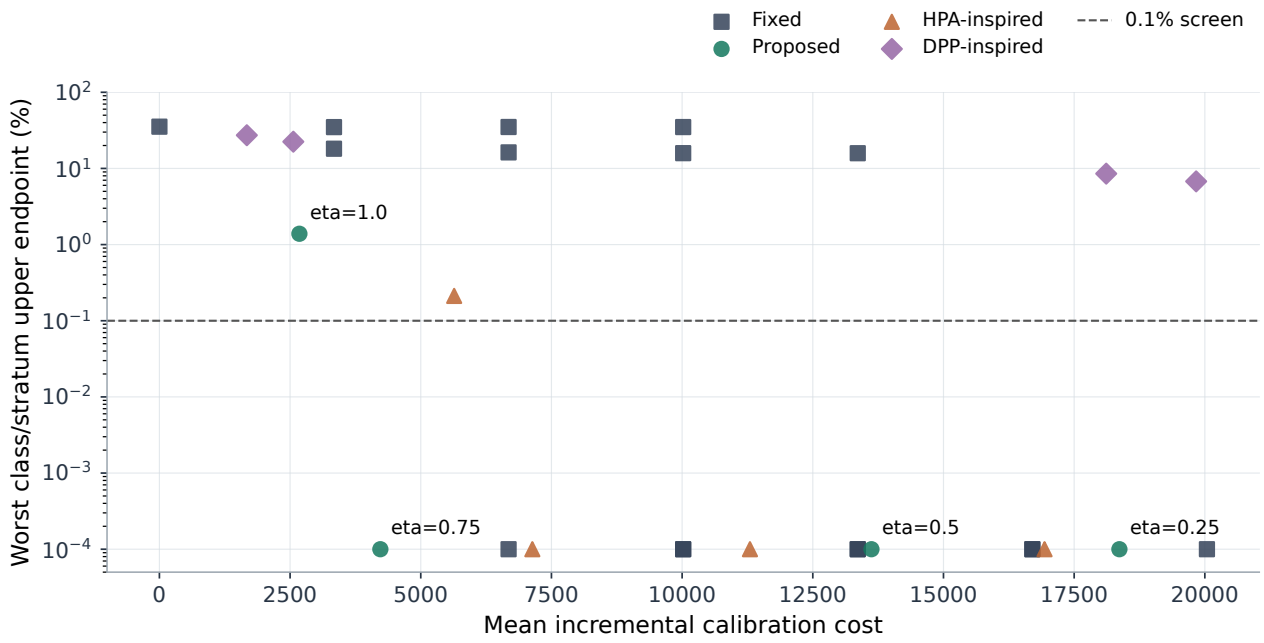


FIGURE 3. Calibration cost and worst class/stratum upper endpoint. Each marker represents one candidate evaluated on twelve runs. The horizontal line is the 0.1% screening target. Points below the display floor represent zero empirical endpoints, not certified zero risk.

TABLE 4. Headroom sensitivity on calibration seeds. Upper endpoints are percentages.

η	Mean cost	Upper V_0	Upper V_1	Eligible
0.25	18365.44	0.0000	0.0000	Yes
0.50	13621.71	0.0000	0.0000	Yes
0.75	4226.92	0.0000	0.0000	Yes
1.00	2678.08	0.7315	1.3879	No

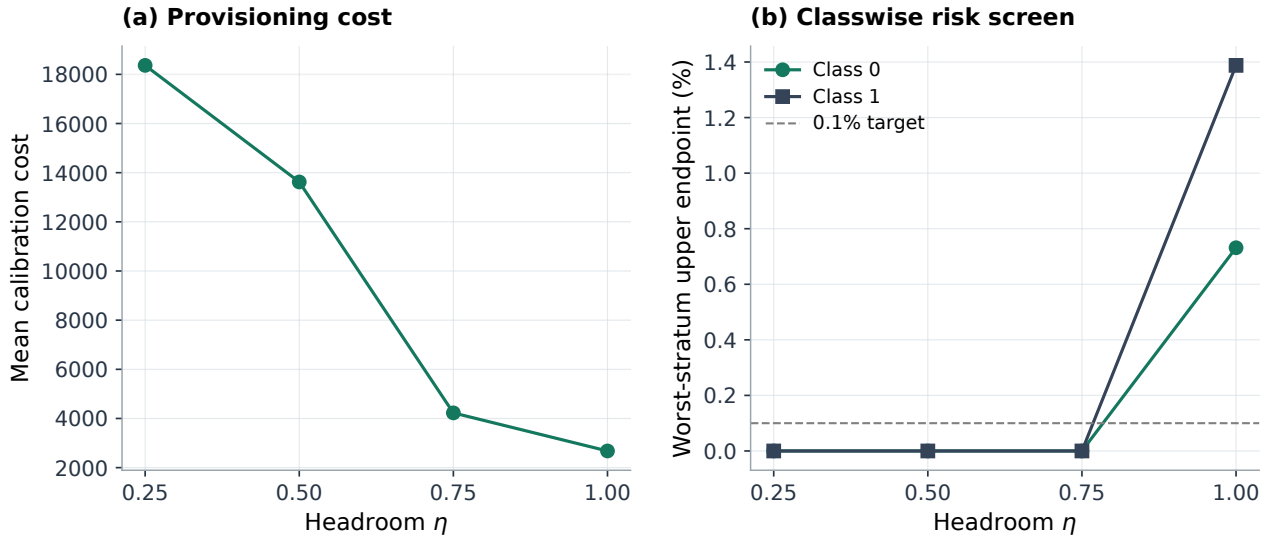


FIGURE 4. Sensitivity to headroom on the frozen calibration grid. Left: mean incremental cost. Right: worst-stratum classwise upper endpoints. The dashed line marks the empirical screening target.

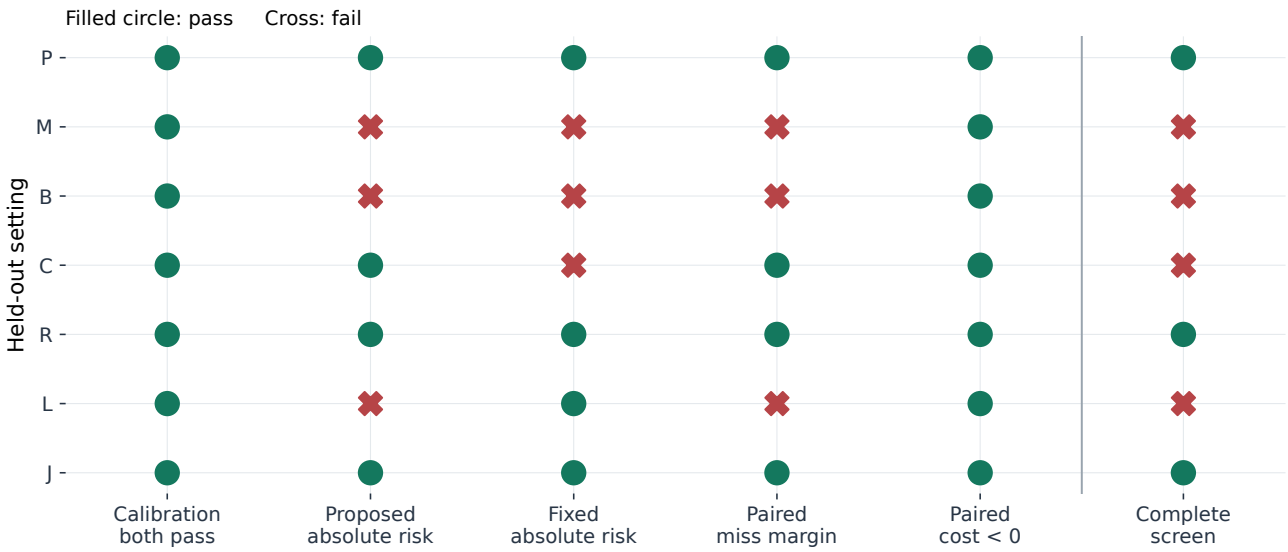


FIGURE 5. Qualification of proposed versus fixed capacity, decomposed by condition.

TABLE 5. Common-target comparison against fixed capacity. Upper endpoints are percentages. Cost intervals are paired, unadjusted 95% intervals.

Setting	Proposed (U_0, U_1)	Fixed (U_0, U_1)	ΔJ [95% interval]	Qualified
P	(0.0000, 0.0000)	(0.0000, 0.0457)	-2520.2 [-2593.1, -2447.0]	Yes
M	(0.2485, 1.9804)	(0.0173, 2.4488)	-2180.8 [-2448.9, -1889.4]	No
B	(0.6058, 0.2601)	(0.2013, 0.7197)	-1402.7 [-1917.4, -844.9]	No
C	(0.0000, 0.0000)	(0.0000, 0.1349)	-2506.4 [-2584.6, -2425.6]	No
R	(0.0331, 0.0000)	(0.0000, 0.0789)	-2478.2 [-2577.8, -2378.0]	Yes
L	(1.1927, 9.8982)	(0.0000, 0.0623)	-2156.4 [-2227.3, -2082.4]	No
J	(0.0000, 0.0062)	(0.0140, 0.0581)	-2378.9 [-2463.9, -2286.0]	Yes

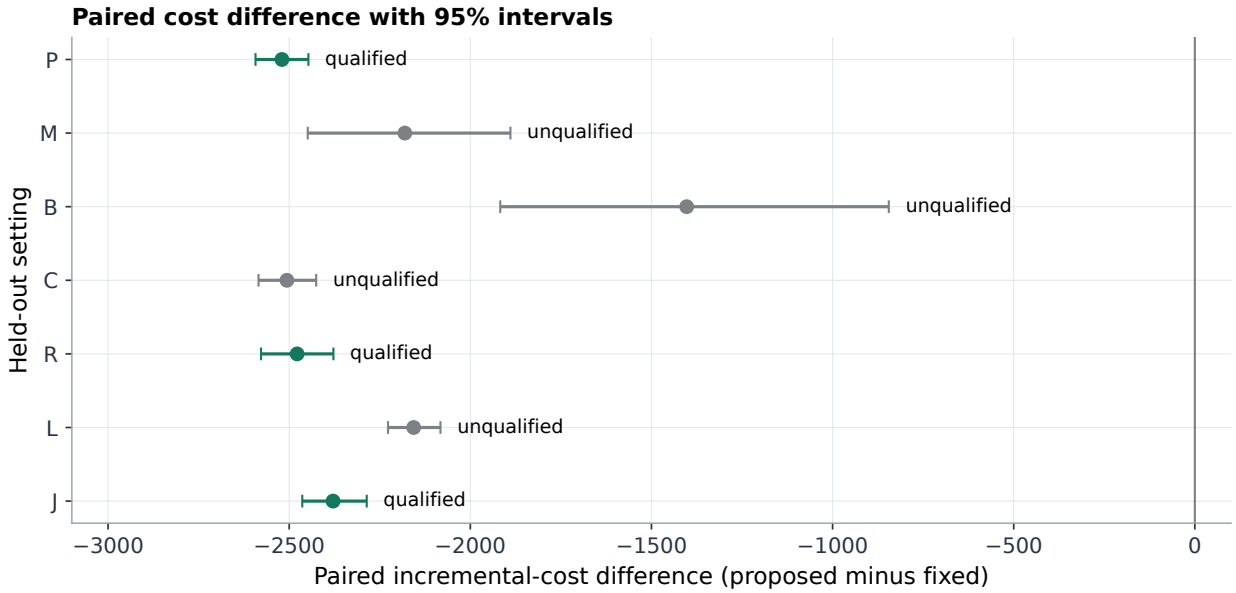


FIGURE 6. Paired cost differences, proposed minus fixed. Green denotes a comparison satisfying the entire descriptive qualification rule. Gray denotes an unqualified comparison, regardless of the direction of its cost difference. Intervals are unadjusted 95% paired-bootstrap intervals.

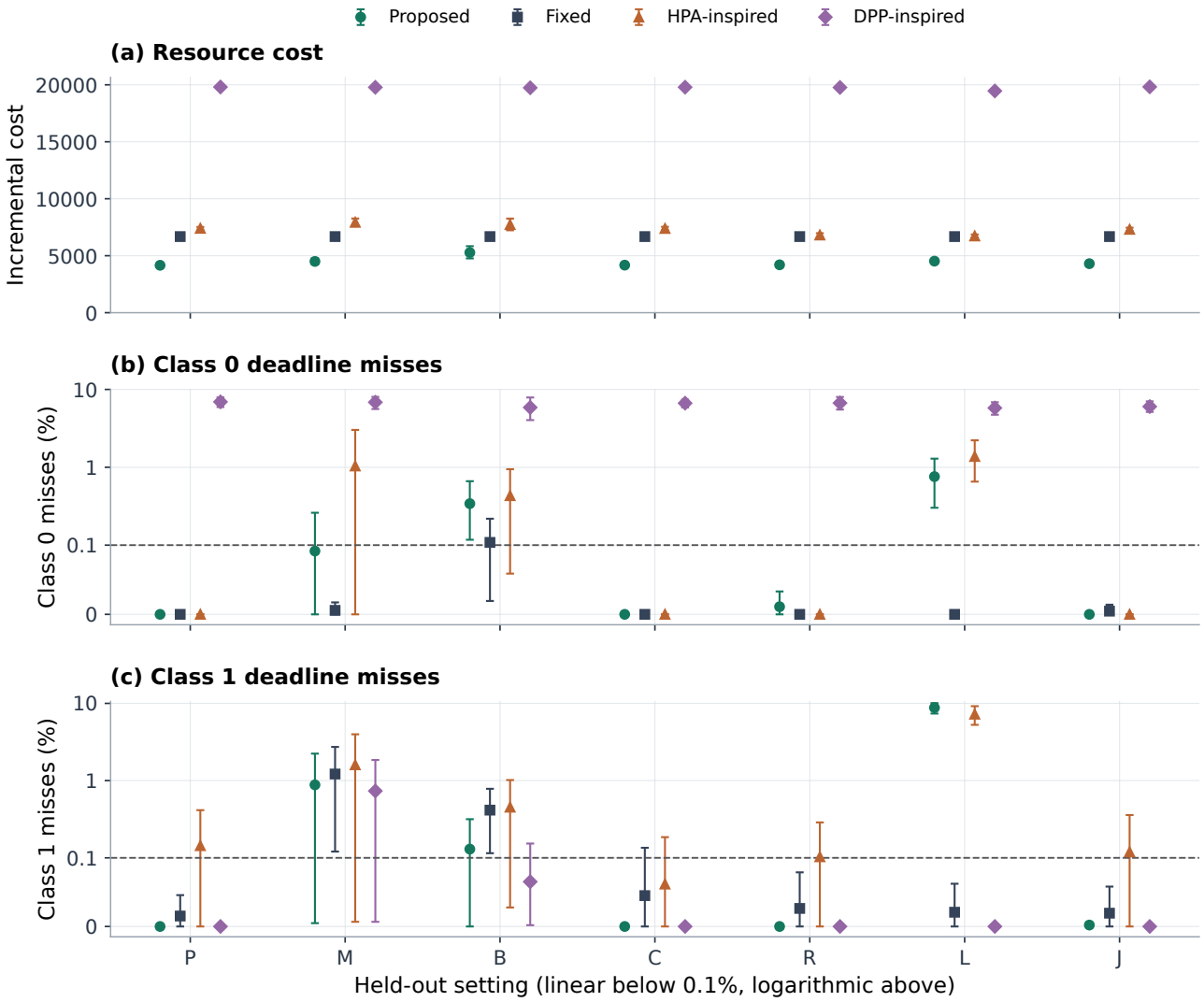


FIGURE 7. **Held-out costs and deadline-miss percentages with unadjusted 95% seed-bootstrap intervals. P, M, B, C, R, L and J are defined in Table 3. The 0.1% dashed line is the common target. The DPP-inspired adaptation is calibration-ineligible and diagnostic only.**

delay on issue. Its successful screen neither contradicts L nor isolates the effect of jitter at equal mean delay.

Figure 8 compares the proposed controller with a rate-only projection at the same headroom and with an uncalibrated ledger variant. In P, the no-ledger cost is 4619.93 versus 4159.85 for the proposed policy, approximately a 9.96% reduction. Both have zero observed misses. The uncalibrated controller is cheaper at 2737.40, but its class means become 0.0570% and 0.1447%. The two ablations answer different questions: ledger availability at fixed headroom, and headroom selection with the ledger present. Their effects should not be merged into one causal attribution. To connect these cost differences to the objective, write

$$J = J_{\mathrm{cap}} + J_{\mathrm{sw}}, \qquad J_{\mathrm{cap}} = \sum_{t,i} e_i(t),$$
$$J_{\mathrm{sw}} = \kappa \sum_{a \in \mathcal{A}_T} \|x_a - e_a^-\|_1. \tag{51}$$

Figure 9 recomputes these terms from the saved per-run counters. The hatched caps in panel (a) show switching, enlarged separately in panel (b). They are not additional experiments or newly selected outcomes.

In P, the ledger variant has mean capacity cost 4100.75 and switching cost 59.10. The no-ledger values are 4551.25 and 68.675. The capacity difference is 450.50, accounting for 97.92% of the total mean cost difference. Thus the nominal saving is primarily reduced active capacity, not merely reduced switching charges.

The ledger is not uniformly beneficial to deadlines. In L, its class-1 mean miss fraction is 8.7846% compared with 3.9807% without the ledger. Neither passes the absolute screen, and the ledger uses less capacity. This is a cost–risk trade-off, not evidence that the ledger is always preferable. The aggregate counters also do not measure forecast error

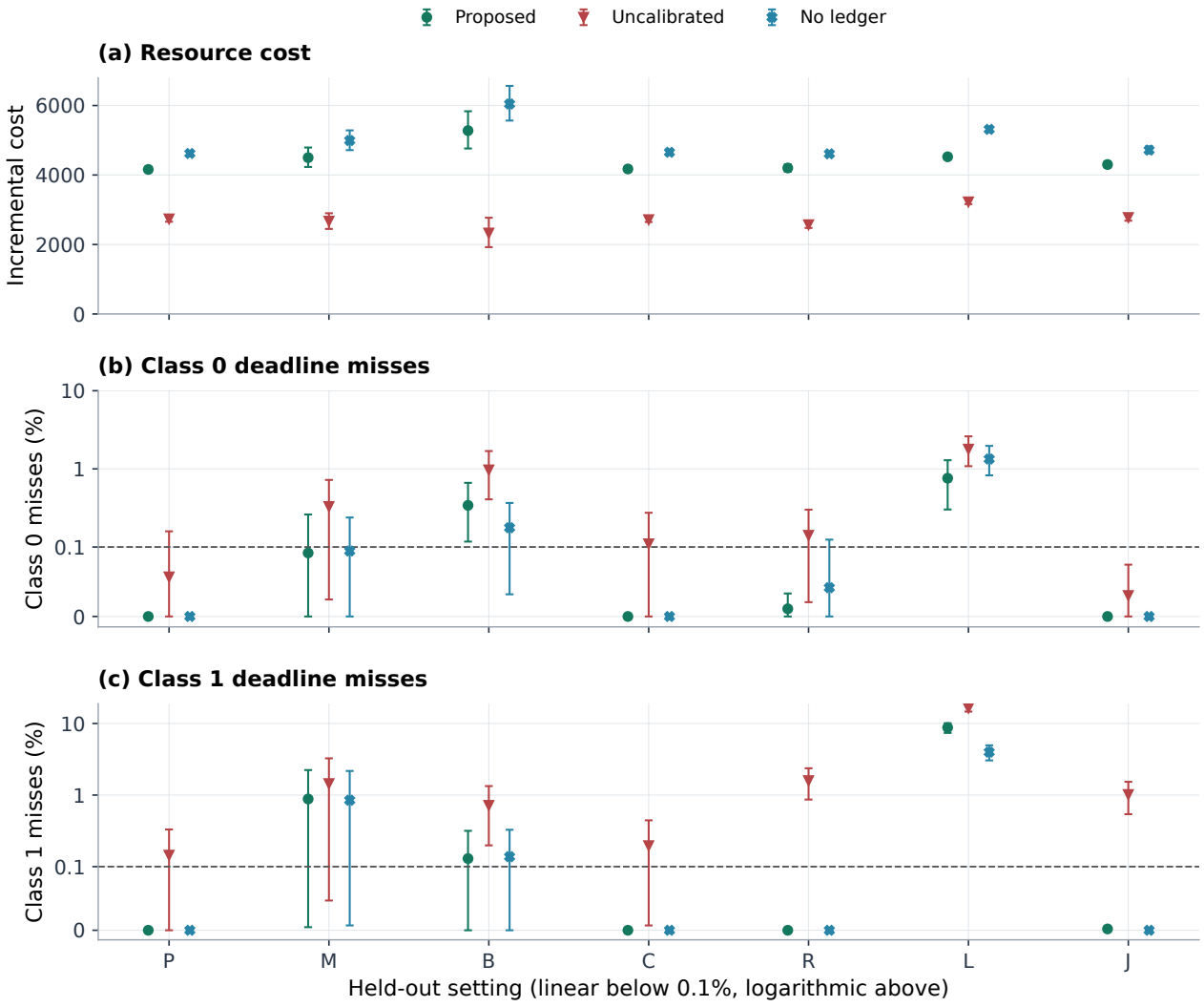


FIGURE 8. Component ablations on identical test seeds. The uncalibrated controller uses $\eta = 1$. The no-ledger controller retains selected headroom but uses a drift projection. The error bars describe seed-mean uncertainty, not predictive confidence bounds for the controller.

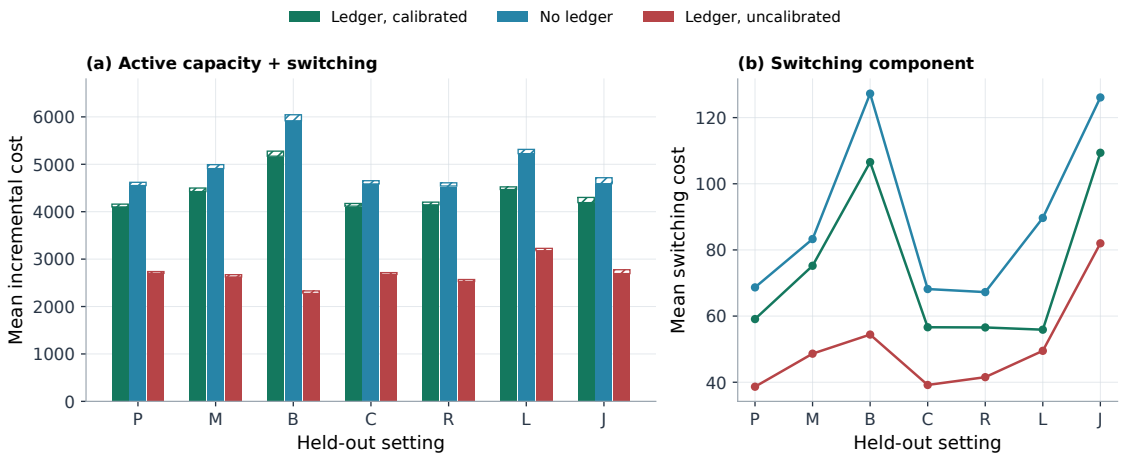


FIGURE 9. Post hoc cost decomposition of the frozen ablation runs. Hatched caps denote switching cost.

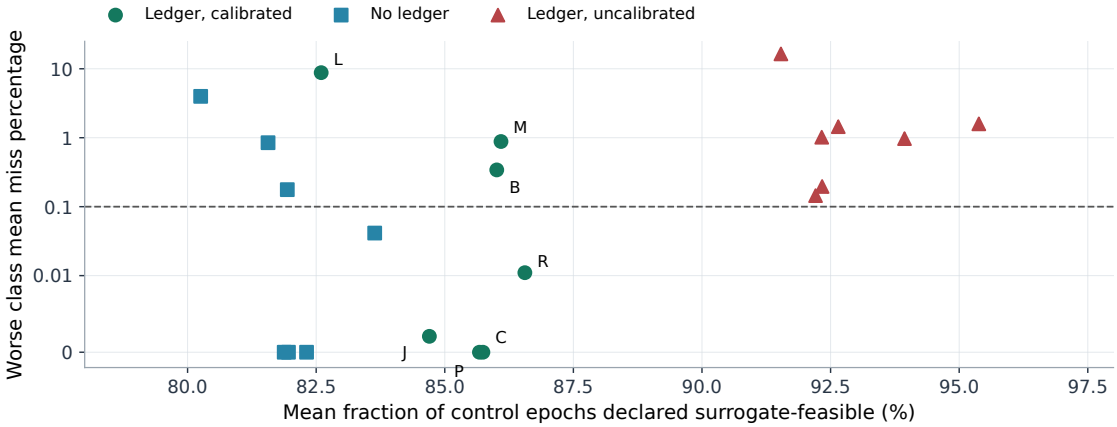


FIGURE 10. **Post hoc proxy–outcome diagnostic. Labels identify proposed-policy settings. The dashed line is 0.1%.**

or explain the timing of every capacity release. Such mechanisms require per-slot traces.

Define the run-level feasible-decision frequency

$$\phi = \frac{1}{N_{\mathrm{ctrl}}} \sum_{t \in \mathcal{T}_{\mathrm{ctrl}}} \mathbf{1}\{\max_{s} \rho_s(x_t, \alpha_t) \le 1\}, \tag{52}$$

where $\mathcal{T}_{\mathrm{ctrl}}$ is the set of control epochs and N_{ctrl} its size. Figure 10 compares the seed-mean ϕ with the worse of the two class seed-mean miss fractions. This exploratory diagnostic uses existing counters. It is not a calibration criterion, a prediction-accuracy test, or an event-level correlation analysis.

The proposed controller declares 85.67% of decisions feasible in P and 86.09% in M, although the worse class mean changes from zero to 0.8811%. In L, 82.60% of decisions remain surrogate-feasible while the worse class mean reaches 8.7846%.

Equations (11)–(13) explain why a positive queue-level margin does not reveal recoverable deadline slack. Likewise, (39)–(40) show that a nominal $\rho_s \le 1$ leaves demand-error exposure unbudgeted. The proxy does not encode packet ages, coupled downstream forecasts or an error budget over the full activation horizon. Consequently, similar values of ϕ can coexist with very different deadline outcomes. Smaller headroom tightens the score, but it does not repair those omitted state variables.

VIII. Discussion and Deployment Boundary

The supported advantage is conditional. P and R test repeatability within the calibration traffic constructions, while J tests one announced-delay law. Failures in M, B and L prevent extrapolation to general workload or latency robustness. Integer reservations, headroom and ledger replay explain how decisions are formed. Their consistency with aggregate results does not identify forecast accuracy or establish packet-level guarantees.

Six seeds per calibration stratum, twenty per test setting, all-zero samples and unadjusted comparisons limit inference at the 0.1% target. The fixed reference has its own share policy, and HPA/DPP are adaptations. Neither competitive-system reproduction nor physical O-RAN validation is provided. The next validation steps are measured service and interface-delay mappings, recorded workloads, independently implemented baselines and larger reliability experiments. Channel variation, retransmissions and virtualization interference must enter that model before interpreting abstract cost as deployment benefit.

IX. Conclusion

The action-ledger controller separates measured, requested and activated capacity. With independently calibrated headroom, it yields qualified cost reductions of 35.61–37.73% against fixed provisioning in three of seven synthetic settings. Its nominal ledger ablation saves approximately 9.96%, primarily through lower active capacity, with zero observed misses for both variants. Shifted traffic and long actuation nevertheless expose deadline failures despite frequent surrogate-feasible decisions. The study provides an auditable controller and identifies the conditions under which known-action replay reduces provisioning cost while satisfying the empirical comparison screen. Physical deployment and rare-event reliability validation remain future work.

Appendix A
Complete Held-Out Results

Table 6 reports every method and setting using the same classwise metrics.

TABLE 6. Complete test results, twenty seeds per row. Miss fractions and one-sided upper endpoints are percentages. DPP-inspired results are diagnostic after calibration failure.

Setting	Method	Mean V_0	Mean V_1	Upper V_0	Upper V_1	Mean cost
P	Proposed	0.0000	0.0000	0.0000	0.0000	4159.85
P	Fixed	0.0000	0.0152	0.0000	0.0457	6680.00
P	HPA-inspired	0.0000	0.1436	0.0000	0.3977	7400.26
P	DPP-inspired	6.9240	0.0000	7.7209	0.0000	19805.75
P	Uncalibrated	0.0570	0.1447	0.1463	0.2973	2737.40
P	No ledger	0.0000	0.0000	0.0000	0.0000	4619.93
M	Proposed	0.0915	0.8811	0.2485	1.9804	4499.21
M	Fixed	0.0058	1.2169	0.0173	2.4488	6680.00
M	HPA-inspired	1.0388	1.6008	2.9262	3.5819	7932.65
M	DPP-inspired	6.8437	0.7340	7.8499	1.5975	19779.96
M	Uncalibrated	0.3303	1.4469	0.6362	2.9848	2671.89
M	No ledger	0.0942	0.8468	0.2135	1.8537	4992.29
B	Proposed	0.3413	0.1297	0.6058	0.2601	5277.30
B	Fixed	0.1084	0.4146	0.2013	0.7197	6680.00
B	HPA-inspired	0.4319	0.4528	0.8366	0.9144	7729.49
B	DPP-inspired	5.8678	0.0652	7.5206	0.1349	19738.71
B	Uncalibrated	0.9710	0.7176	1.5660	1.2352	2329.66
B	No ledger	0.1766	0.1378	0.3322	0.2777	6044.76
C	Proposed	0.0000	0.0000	0.0000	0.0000	4173.62
C	Fixed	0.0000	0.0450	0.0000	0.1349	6680.00
C	HPA-inspired	0.0000	0.0618	0.0000	0.1853	7388.18
C	DPP-inspired	6.6611	0.0000	7.3007	0.0000	19790.64
C	Uncalibrated	0.1099	0.1963	0.2439	0.4068	2715.18
C	No ledger	0.0000	0.0000	0.0000	0.0000	4653.68
R	Proposed	0.0110	0.0000	0.0331	0.0000	4201.81
R	Fixed	0.0000	0.0263	0.0000	0.0789	6680.00
R	HPA-inspired	0.0000	0.1031	0.0000	0.2656	6814.40
R	DPP-inspired	6.6940	0.0000	7.7193	0.0000	19766.59
R	Uncalibrated	0.1412	1.5900	0.2721	2.2523	2569.54
R	No ledger	0.0414	0.0000	0.1243	0.0000	4609.25
L	Proposed	0.7593	8.7846	1.1927	9.8982	4523.64
L	Fixed	0.0000	0.0208	0.0000	0.0623	6680.00
L	HPA-inspired	1.3767	7.2461	2.0694	8.8912	6731.15
L	DPP-inspired	5.7789	0.0000	6.6497	0.0000	19461.90
L	Uncalibrated	1.7900	16.4355	2.4647	17.7445	3226.75
L	No ledger	1.3369	3.9807	1.8522	4.8110	5314.18
J	Proposed	0.0000	0.0021	0.0000	0.0062	4301.07
J	Fixed	0.0047	0.0194	0.0140	0.0581	6680.00
J	HPA-inspired	0.0000	0.1194	0.0000	0.3582	7312.54
J	DPP-inspired	6.0246	0.0000	6.8568	0.0000	19819.75
J	Uncalibrated	0.0302	1.0178	0.0636	1.4476	2776.40
J	No ledger	0.0000	0.0000	0.0000	0.0000	4716.75

Appendix B
Complete Paired Comparisons

Table 7 reports cost and miss differences for every comparator.

TABLE 7. All paired contrasts, proposed minus comparator. Miss differences and upper endpoints are percentage points. No multiplicity-adjusted claim is made.

Setting	Comparator	ΔJ [95% interval]	Mean $(\Delta V_0, \Delta V_1)$	Upper $(\Delta V_0, \Delta V_1)$
P	Fixed	-2520.2 [-2593.1, -2447.0]	(0.0000, -0.0152)	(0.0000, 0.0000)
P	HPA-inspired	-3240.4 [-3351.1, -3130.5]	(0.0000, -0.1436)	(0.0000, 0.0000)
P	DPP-inspired	-15645.9 [-15721.6, -15569.6]	(-6.9240, 0.0000)	(-6.1061, 0.0000)
P	Uncalibrated	1422.5 [1357.5, 1490.1]	(-0.0570, -0.1447)	(0.0000, -0.0284)
P	No ledger	-460.1 [-510.0, -410.2]	(0.0000, 0.0000)	(0.0000, 0.0000)
M	Fixed	-2180.8 [-2448.9, -1889.4]	(0.0857, -0.3358)	(0.2312, -0.1143)
M	HPA-inspired	-3433.4 [-3647.5, -3181.9]	(-0.9473, -0.7197)	(0.0015, 0.0200)
M	DPP-inspired	-15280.8 [-15559.1, -14975.8]	(-6.7522, 0.1471)	(-5.7626, 0.3231)
M	Uncalibrated	1827.3 [1718.1, 1944.4]	(-0.2388, -0.5658)	(-0.0195, -0.0159)
M	No ledger	-493.1 [-586.8, -403.3]	(-0.0028, 0.0344)	(0.0924, 0.1165)
B	Fixed	-1402.7 [-1917.4, -844.9]	(0.2329, -0.2850)	(0.4310, -0.1079)
B	HPA-inspired	-2452.2 [-2614.5, -2295.6]	(-0.0905, -0.3231)	(0.0997, -0.0685)
B	DPP-inspired	-14461.4 [-14977.8, -13899.2]	(-5.5265, 0.0645)	(-4.1229, 0.1515)
B	Uncalibrated	2947.6 [2710.2, 3175.2]	(-0.6297, -0.5879)	(-0.2582, -0.2014)
B	No ledger	-767.5 [-839.8, -690.4]	(0.1647, -0.0081)	(0.3303, 0.0000)
C	Fixed	-2506.4 [-2584.6, -2425.6]	(0.0000, -0.0450)	(0.0000, 0.0000)
C	HPA-inspired	-3214.6 [-3358.2, -3071.5]	(0.0000, -0.0618)	(0.0000, 0.0000)
C	DPP-inspired	-15617.0 [-15710.2, -15526.1]	(-6.6611, 0.0000)	(-5.9980, 0.0000)
C	Uncalibrated	1458.5 [1393.4, 1520.2]	(-0.1099, -0.1963)	(-0.0062, -0.0326)
C	No ledger	-480.1 [-544.8, -417.0]	(0.0000, 0.0000)	(0.0000, 0.0000)
R	Fixed	-2478.2 [-2577.8, -2378.0]	(0.0110, -0.0263)	(0.0331, 0.0000)
R	HPA-inspired	-2612.6 [-2782.0, -2432.3]	(0.0110, -0.1031)	(0.0331, 0.0000)
R	DPP-inspired	-15564.8 [-15669.4, -15463.9]	(-6.6830, 0.0000)	(-5.6565, 0.0000)
R	Uncalibrated	1632.3 [1541.1, 1726.6]	(-0.1302, -1.5900)	(-0.0172, -0.9720)
R	No ledger	-407.4 [-476.0, -341.8]	(-0.0304, 0.0000)	(0.0221, 0.0000)
L	Fixed	-2156.4 [-2227.3, -2082.4]	(0.7593, 8.7638)	(1.1927, 9.8689)
L	HPA-inspired	-2207.5 [-2330.2, -2083.7]	(-0.6174, 1.5385)	(0.0462, 3.0453)
L	DPP-inspired	-14938.3 [-15000.8, -14871.3]	(-5.0197, 8.7846)	(-4.2307, 9.8982)
L	Uncalibrated	1296.9 [1217.8, 1376.6]	(-1.0308, -7.6509)	(-0.5847, -6.6234)
L	No ledger	-790.5 [-874.8, -698.8]	(-0.5776, 4.8039)	(-0.0554, 5.5754)
J	Fixed	-2378.9 [-2463.9, -2286.0]	(-0.0047, -0.0173)	(0.0000, 0.0000)
J	HPA-inspired	-3011.5 [-3146.5, -2883.1]	(0.0000, -0.1173)	(0.0000, 0.0042)
J	DPP-inspired	-15518.7 [-15624.6, -15403.8]	(-6.0246, 0.0021)	(-5.2649, 0.0062)
J	Uncalibrated	1524.7 [1446.8, 1604.0]	(-0.0302, -1.0158)	(0.0000, -0.6151)
J	No ledger	-415.7 [-478.1, -355.4]	(0.0000, 0.0021)	(0.0000, 0.0062)

Appendix C
Reproducibility

Run `python reliability_study.py` to reproduce the calibration and held-out matrix. Existing complete checkpoints are reused only if the frozen protocol hash matches. Remove or relocate those specific checkpoint files to execute all runs anew. Run `python verify_reliability.py` to reconcile all metrics and selected deterministic replays, followed by `python build_reliability_paper.py` and `python build_release.py`. The archive includes the exact dependency versions and checksums. The manuscript tables and plots are generated from the saved execution records. The repository also includes vector schematics, figure previews and their rendering script.

Data and Code Availability

The archive provides code, the frozen protocol, all 1176 execution records, verification tests, manuscript sources and vector figures. All reported diagnostics use the same execution records. The workloads are generated synthetically from the documented seeds and distributions. The code, frozen execution records and reproduction instructions are available at github.com/YassirALKarawi/oran-telemetry-elasticity.

REFERENCES

[1] M. Polese et al., "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges," IEEE Communications Surveys & Tutorials, vol. 25, no. 2, pp. 1376–1411, 2023. 10.1109/COMST.2023.3239220.

[2] D. Kreutz et al., "Software-Defined Networking: A Comprehensive Survey," Proceedings of the IEEE, vol. 103, no. 1, pp. 14–76, 2015. 10.1109/JPROC.2014.2371999.

[3] A. Garcia-Saavedra and X. Costa-Perez, "O-RAN: Disrupting the Virtualized RAN Ecosystem," IEEE Communications Standards Magazine, vol. 5, no. 4, pp. 96–103, 2021. 10.1109/MCOMSTD.101.2000014.

[4] R. D. Yates et al., "Age of Information: An Introduction and Survey," IEEE Journal on Selected Areas in Communications, vol. 39, no. 5, pp. 1183–1210, 2021. 10.1109/JSAC.2021.3065072.

[5] J. P. Hespanha, P. Naghshtabrizi, and Y. Xu, "A Survey of Recent Results in Networked Control Systems," Proceedings of the IEEE, vol. 95, no. 1, pp. 138–162, 2007. 10.1109/JPROC.2006.887288.

[6] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, "Constrained model predictive control: Stability and optimality," Automatica, vol. 36, no. 6, pp. 789–814, 2000. 10.1016/S0005-1098(99)00214-9.

[7] S. D'Oro, L. Bonati, M. Polese, and T. Melodia, "OrchestRAN: Network Automation through Orchestrated Intelligence in the Open RAN," IEEE INFOCOM, 2022, author version arXiv:2201.05632. 10.48550/arXiv.2201.05632.

[8] F. Mungari, C. Puligheddu, A. Garcia-Saavedra, and C. F. Chiasserini, "O-RAN Intelligence Orchestration Framework for Quality-Driven xApp Deployment and Sharing," IEEE Transactions on Mobile Computing, vol. 24, no. 6, pp. 4811–4828, 2025. 10.1109/TMC.2025.3527707.

[9] O. T. Demir, M. Masoudi, E. Bjornson, and C. Cavdar, "Cell-Free Massive MIMO in O-RAN: Energy-Aware Joint Orchestration of Cloud, Fronthaul, and Radio Resources," IEEE Journal on Selected Areas in Communications, vol. 42, no. 2, pp. 356–372, 2024. 10.1109/JSAC.2023.3336187.

[10] J. A. Ayala-Romero et al., "vrAIn: A Deep Learning Approach Tailoring Computing and Radio Resources in Virtualized RANs," ACM MobiCom, 2019. 10.1145/3300061.3345431.

[11] G. Garcia-Aviles et al., "Nuberu: Reliable RAN virtualization in shared platforms," ACM MobiCom, 2021. 10.1145/3447993.3483266.

[12] X. Foukas and B. Radunovic, "Concordia: Teaching the 5G vRAN to Share Compute," ACM SIGCOMM, 2021. Author/publisher source.

[13] M. Kalntis, G. Iosifidis, and F. A. Kuipers, "Adaptive Resource Allocation for Virtualized Base Stations in O-RAN With Online Learning," IEEE Transactions on Communications, vol. 73, no. 3, pp. 1787–1800, 2025. 10.1109/TCOMM.2024.3461569.

[14] O. Adamuz-Hinojosa, L. Zanzi, V. Sciancalepore, and X. Costa-Perez, "MAREA: A Delay-Aware Multi-Time-Scale Radio Resource Orchestrator for 6G O-RAN," IEEE Transactions on Communications, vol. 73, no. 9, pp. 7695–7710, 2025. 10.1109/TCOMM.2025.3552296.

[15] S. D. A. Shah, M. Hafeez, A. Salama, and S. A. R. Zaidi, "Proactive AI-and-RAN Workload Orchestration in O-RAN Architectures for 6G Networks," author version, arXiv:2507.09124, 2025. 10.48550/arXiv.2507.09124.

[16] F. W. Murti, S. Ali, G. Iosifidis, and M. Latva-aho, "Learning-Based Orchestration for Dynamic Functional Split and Resource Allocation in vRANs," EuCNC/6G Summit, 2022. 10.1109/EuCNC/6GSummit54941.2022.9815815.

[17] Q. Wang et al., "Resource Allocation Based on Radio Intelligence Controller for Open RAN Toward 6G," IEEE Access, vol. 11, pp. 97909–97919, 2023. 10.1109/ACCESS.2023.3311888.

[18] V. Kasuluru et al., "Cloud-Native Multivariate Probabilistic Forecasting for Next-Generation Resource Allocation in O-RAN," IEEE Access, vol. 14, pp. 2904–2923, 2026. 10.1109/ACCESS.2025.3648065.

[19] A. Tuerxun and A. Nakao, "SORA: Energy-Efficient Resource Allocation in Open Radio Access Network With Online Learning," IEEE Access, vol. 13, pp. 113686–113701, 2025. 10.1109/ACCESS.2025.3584345.

[20] A. E. Giannopoulos et al., "COMIX: Generalized Conflict Management in O-RAN xApps—Architecture, Workflow, and a Power Control Case," IEEE Access, vol. 13, pp. 116684–116700, 2025. 10.1109/ACCESS.2025.3585774.

[21] M. Iqbal and A. Nakao, "5ERASI: 5G Service-Enabled Resource Allocation and Sub-Slice Isolation for Open RAN," IEEE Access, 2026. 10.1109/ACCESS.2026.3693783.

[22] Y. Chen, R. Yao, H. Hassanieh, and R. Mittal, "Channel-Aware 5G RAN Slicing with Customizable Schedulers," USENIX NSDI, pp. 1767–1782, 2023. Author/publisher source.

[23] W.-H. Ko et al., "EdgeRIC: Empowering Real-time Intelligent Optimization and Control in NextG Cellular Networks," USENIX NSDI, pp. 1315–1330, 2024. Author/publisher source.

[24] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "ColO-RAN: Developing Machine Learning-based xApps for Open RAN Closed-loop Control on Programmable Experimental Platforms," IEEE Transactions on Mobile Computing, vol. 22, no. 10, pp. 5787–5800, 2023. 10.1109/TMC.2022.3188013.

[25] L. Bonati, M. Polese, S. D'Oro, S. Basagni, and T. Melodia, "Open-RAN Gym: AI/ML Development, Data Collection, and Testing for O-RAN on PAWR Platforms," Computer Networks, vol. 220, article 109502, 2023. 10.1016/j.comnet.2022.109502.

[26] L. Bonati, S. D'Oro, S. Basagni, and T. Melodia, "SCOPE: An Open and Softwarized Prototyping Platform for NextG Systems," ACM MobiSys, 2021. 10.1145/3458864.3466863.

[27] R. Schmidt, M. Irazabal, and N. Nikaein, "FlexRIC: an SDK for next-generation SD-RANs," ACM CoNEXT, pp. 411–425, 2021. 10.1145/3485983.3494870.

[28] A. Lacava et al., "ns-O-RAN: Simulating O-RAN 5G Systems in ns-3," ACM Workshop on ns-3, 2023. 10.1145/3592149.3592161.

[29] M. Polese et al., "Colosseum: The Open RAN Digital Twin," IEEE Open Journal of the Communications Society, vol. 5, pp. 5452–5466, 2024. 10.1109/OJCOMS.2024.3447472.

[30] N. Patriciello, S. Lagen, B. Bojovic, and L. Giupponi, "An E2E Simulator for 5G NR Networks," Simulation Modelling Practice and Theory, vol. 96, article 101933, 2019. Author/publisher source.

[31] K. Koutlia, B. Bojovic, Z. Ali, and S. Lagen, "Calibration of the 5G-LENA System Level Simulator in 3GPP reference scenarios,"

Simulation Modelling Practice and Theory, vol. 119, article 102580, 2022. Author/publisher source.

[32] M. J. Neely, "Stochastic Network Optimization with Application to Communication and Queueing Systems," Morgan & Claypool, 2010, Springer electronic edition. 10.1007/978-3-031-79995-2.

[33] J.-Y. Le Boudec and P. Thiran, "Network Calculus: A Theory of Deterministic Queuing Systems for the Internet," Springer, 2001. 10.1007/3-540-45318-0.

[34] Y. Jiang and Y. Liu, "Stochastic Network Calculus," Springer, 2008. 10.1007/978-1-84800-127-5.

[35] S. M. Tabatabaee, J.-Y. Le Boudec, and M. Boyer, "Interleaved Weighted Round-Robin: A Network Calculus Analysis," IEICE Transactions on Communications, vol. E104-B, no. 12, pp. 1479–1493, 2021. 10.1587/transcom.2021ITI0001.

[36] S. M. Tabatabaee and J.-Y. Le Boudec, "Deficit Round-Robin: A Second Network Calculus Analysis," IEEE/ACM Transactions on Networking, vol. 30, no. 5, pp. 2216–2230, 2022. 10.1109/TNET.2022.3164772.

[37] E. Mohammadpour, E. Stai, and J.-Y. Le Boudec, "Improved Network-Calculus Nodal Delay-Bounds in Time-Sensitive Networks," IEEE/ACM Transactions on Networking, vol. 31, no. 6, pp. 2902–2917, 2023. 10.1109/TNET.2023.3275910.

[38] R. G. Sargent, "Verification and validation of simulation models," Journal of Simulation, vol. 7, pp. 12–24, 2013. 10.1057/jos.2012.20.

[39] Kubernetes Authors, "Horizontal Pod Autoscaling," official documentation, accessed October 22, 2026. Author/publisher source.

[40] M. J. Neely, "Energy-Aware Wireless Scheduling with Near Optimal Backlog and Convergence Time Tradeoffs," author version, arXiv:1411.4740, 2014. 10.48550/arXiv.1411.4740.

[41] J. Wang, B. Liang, M. Dong, G. Boudreau, and H. Abouzeid, "Delay-Tolerant Constrained OCO with Application to Network Resource Allocation," IEEE INFOCOM, 2021. 10.1109/INFOCOM42981.2021.9488698.

[42] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," The Annals of Statistics, vol. 7, no. 1, pp. 1–26, 1979. 10.1214/aos/1176344552.

[43] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," Scandinavian Journal of Statistics, vol. 6, pp. 65–70, 1979. Author/publisher source.